WUOLAH

POO PRACTICAS COMPLETAS - NOTA 1...

RaizDeX

Programación Orientada a Objetos

2º Grado en Ingeniería Informática

Escuela Superior de Ingeniería
Universidad de Cádiz

Reservados todos los derechos.
No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

POO: PRÁCTICAS COMPLETAS

Curso 2020-21

Escuela Superior de Ingeniería. Universidad de Cádiz
Grado en Ingeniería Informática

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Índice de Contenidos

Introducción 1
Práctica 0 2
- Fecha.hpp 2
- Fecha.cpp 4
- Cadena.hpp 8
- Cadena.cpp 10
- Makefile 13

Práctica 1 14
- Fecha.hpp 14
- Fecha.cpp 16
- Cadena.hpp 21
- Cadena.cpp 24
- Makefile 28

Practica 2 29
- Fecha.hpp 29
- Fecha.cpp 31
- Cadena.hpp 36
- Cadena.cpp 40
- Makefile (P1) 44
- Articulo.hpp 45
- Articulo.cpp 47
- Tarjeta.hpp 48
- Tarjeta.cpp 52
- Usuario.hpp 55
- Usuario.cpp 58

Práctica 3 61
- Fecha.hpp 61
- Fecha.cpp 63
- Cadena.hpp 68
- Cadena.cpp 72
- Makefile (P1) 76
- Articulo.hpp 77
- Articulo.cpp 79

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Tarjeta.hpp 80
·Tarjeta.cpp 84
·Usuario.hpp 87
·Usuario.cpp 90
·Usuario-Pedido.hpp 93
·Usuario-Pedido.cpp 94
·Pedido-Articulo.hpp 95
·Pedido-Articulo.cpp 97
Práctica 4 100
·Fecha.hpp 100
·Fecha.cpp 102
·Cadena.hpp 107
·Cadena.cpp 111
·Makefile (P1) 115
·Articulo.hpp 116
·Articulo.cpp 120
·Tarjeta.hpp 121
·Tarjeta.cpp 125
·Usuario.hpp 128
·Usuario.cpp 131
·Pedido.hpp 134
·Pedido.cpp 136
·Usuario-Pedido.hpp 138
·Usuario-Pedido.cpp 139
·Pedido-Articulo.hpp 140
·Pedido-Articulo.cpp 142

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Introducción:

AVISO IMPORTANTE: Este documento contiene todo el código de las prácticas correspondientes al curso 2020-2021, y pasa todas las pruebas de este año (Me pusieron un 10 en las prácticas). Sin embargo, si lo estás haciendo **en años posteriores algunas partes del enunciado de la práctica podrían haber cambiado**, por lo que solo este código puede que no te sirva para tener las prácticas bien. Eso no quita que puedas partir de este código y realizar las modificaciones necesarias o utilizar este documento como guía, y realizar tu propia solución (lo más recomendable para aprobar el examen final de teoría). En cualquier caso, recuerda **pasar todos los tests** para comprobar que todo lo que saques de aquí te sirva.

En las prácticas de POO, tendrás que hacer 5 entregas (numeradas del 0 al 4), que te irán pidiendo a lo largo del curso. Cada práctica consistirá en crear una serie de archivos que luego tendrás que enviar en un .tar.gz. Entre prácticas puede haber modificaciones en archivos que ya hayas entregado, es decir, uno de los ficheros de la P0 que se use en la P1 y "no se haya modificado" en esa práctica puede que sí haya cambiado porque así lo pedía el enunciado, así que debes tener cuidado en no solo copiar los archivos nuevos.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Práctica 0:

·Fecha.hpp:

```
#ifndef _FECHA_HPP_
#define _FECHA_HPP_


class Fecha{
    public:
        //Constructores
        explicit Fecha(int d = 0, int m = 0, int a = 0); //Con 3, 2, 1 y
ningún parámetro
        Fecha(const Fecha& f) = default; //Ctor. de copia
        Fecha(const char* c); //A partir de una cadena de caracteres de f
ormato dd/mm/aaaa
        Fecha& operator = (const Fecha&) = default; //Operador asignación

        //Conversion a const *char
        operator const char*() const;

        //Constantes
        static const int AnnoMinimo = 1902;
        static const int AnnoMaximo = 2037;

        //Clase Fecha Invalida
        class Invalida{
            public:
                Invalida(const char* inf);
                const char* por_que() const;
            private:
                const char* info_;
        };

        //Observadores
        int dia() const noexcept;
        int mes() const noexcept;
        int anno() const noexcept;
        //Operadores
        Fecha& operator ++();
        Fecha& operator ++(int);
        Fecha& operator --();
        Fecha& operator --(int);
        Fecha& operator +=(int i);
        Fecha& operator -=(int i);

        //Operadores Aritméticos
        Fecha operator +(int i) const;
        Fecha operator -(int i) const;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        //Destructor
        ~Fecha() = default;

    private:
        int dia_, mes_, anno_;
        void comprueba();
        int ultimo_dia() const;

};



//Operadores Lógicos
bool operator ==(const Fecha& f1, const Fecha& f2);
bool operator !=(const Fecha& f1, const Fecha& f2);
bool operator <(const Fecha& f1, const Fecha& f2);
bool operator >(const Fecha& f1, const Fecha& f2);
bool operator <=(const Fecha& f1, const Fecha& f2);
bool operator >=(const Fecha& f1, const Fecha& f2);


//Métodos Inline-------------------------------------------------------
-------------------------
inline const char* Fecha::Invalida::por_que() const{return info_;}

inline int Fecha::dia() const noexcept{return dia_;}
inline int Fecha::mes() const noexcept{return mes_;}
inline int Fecha::anno() const noexcept{return anno_;}


#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Fecha.cpp:

```
#include "fecha.hpp"
#include <ctime>
#include <cstdio>
#include <iostream>
#include <locale>

//FUNCIONES PRIVADAS ----------------------------------------------------
-----------------------------
void Fecha::comprueba(){

    if(dia_ > ultimo_dia() || dia_ < 1){ //ultimo dia del mes de la fecha
 paramtero implicito
        Fecha::Invalida d_inval("Dia Incorrecto");
        throw d_inval;
    }
    if(anno_ < Fecha::AnnoMinimo || anno_ > Fecha::AnnoMaximo){
        Fecha::Invalida a_inval("Año Incorrecto");
        throw a_inval;
    }

}
int Fecha::ultimo_dia() const {

    if(mes_ == 1 || mes_ == 3 || mes_ == 5 || mes_ == 7 || mes_ == 8 || m
es_ == 10 || mes_ == 12)
        return 31;

    else if(mes_ == 4 || mes_ == 6 || mes_ == 9 || mes_ == 11)
        return 30;

    else if(mes_ == 2){ //¿Es bisiesto?
        if(anno_ % 4 == 0 && (anno_ % 400 == 0 || anno_ % 100 != 0))
            return 29;
        else
            return 28;

    }else{
        Fecha::Invalida inval("Mes Incorrecto");
        throw inval;
        return 0;
    }
}

//CONSTRUCTORES----------------------------------------------------------
-----------------------------
Fecha::Fecha(int d, int m, int a): dia_{d}, mes_{m}, anno_{a}{
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    std::time_t tiempo_calendario = std::time(nullptr);
    std::tm* tiempo_descompuesto = std::localtime(&tiempo_calendario);
    //Cambio de 0 a día, mes o año actual
    if(anno_ == 0)
        anno_ = tiempo_descompuesto->tm_year +1900;

    if(mes_ == 0)
        mes_ = tiempo_descompuesto->tm_mon + 1;

    if (dia_ == 0)
        dia_ = tiempo_descompuesto->tm_mday;
    comprueba();
}

Fecha::Fecha (const char* c){
    if(sscanf(c,"%d/%d/%d",&dia_,&mes_,&anno_) != 3){
    //Si es distinto de 3, no hemos leído día, mes y año (Falta uno de el
los o hay otros caracteres distintos de '/')
        Fecha::Invalida format("Formato de la cadena incorrecto.");
        throw format;
    }
    std::time_t tiempo_calendario = std::time(nullptr);
    std::tm* tiempo_descompuesto = std::localtime(&tiempo_calendario);

    if(anno_ == 0)
        anno_ = tiempo_descompuesto->tm_year +1900;

    if(mes_ == 0)
        mes_ = tiempo_descompuesto->tm_mon + 1;

    if (dia_ == 0)
        dia_ = tiempo_descompuesto->tm_mday;
    comprueba();
}


//CONVERSOR FECHA ---> CONST CHAR* ------------------------------------
-------------------------
Fecha::operator const char*() const{
    std::locale::global(std::locale("es_ES.utf8"));
    char *aux = new char[100];

    std::time_t tiempo_calendario = std::time(nullptr);
    std::tm* f = std::localtime(&tiempo_calendario);

    f->tm_mday = dia_;
    f->tm_mon = mes_ - 1;
    f->tm_year = anno_ - 1900;
    mktime(f);
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    strftime(aux,100,"%A %e de %B de %Y",f);
    return aux;

}

//OPERADORES-------------------------------------------------------------
-------------------------
Fecha& Fecha::operator +=(int i){
    //Creamos una estructura con la fecha ya operada con el entero y a fo
rmateamos
    std::tm f{};
    f.tm_mday = dia_ + i;
    f.tm_mon = mes_ - 1;
    f.tm_year = anno_ - 1900;
    std::mktime(&f);

    //Pasamos la fecha formateada a los atributos
    dia_ = f.tm_mday;
    mes_ = f.tm_mon + 1;
    anno_ = f.tm_year + 1900;
    comprueba();
    return *this;
}

Fecha& Fecha::operator -=(int i){
    *this += -i;
    return *this;
}

Fecha& Fecha::operator ++(){ //++f

    *this += 1;
    return *this;
}

Fecha& Fecha::operator ++(int){ //f++
    Fecha *f = new Fecha(*this);
    //*f = *this;
    *this += 1;
    return *f;
}

Fecha& Fecha::operator --(){ //--f
    *this += -1;
    return *this;
}

Fecha& Fecha::operator --(int){ //f--
    //Fecha *f = new Fecha();
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    Fecha *f = new Fecha(*this);
    *this += -1;
    return *f;
}

//OPERADORES ARTIMÉTICOS-----------------------------------------------
-------------------------
Fecha Fecha::operator +(int i) const{
    Fecha f(*this);
    return f += i;
}

Fecha Fecha::operator -(int i) const{
    Fecha f(*this);
    return f += -i;
}

//OPERADORES LÓGICOS---------------------------------------------------
-------------------------
bool operator ==(const Fecha& f1, const Fecha& f2){
    return (f1.dia() == f2.dia() && f1.mes() == f2.mes() && f1.anno() ==
f2.anno());
}
bool operator !=(const Fecha& f1, const Fecha& f2){
    return !(f1 == f2);
}
bool operator <(const Fecha& f1, const Fecha& f2){
    return (f1.anno() < f2.anno() || (f1.anno() == f2.anno() && (f1.mes()
 < f2.mes() || (f1.mes() == f2.mes() && f1.dia() < f2.dia()))));
}
bool operator >(const Fecha& f1, const Fecha& f2){
    return f2 < f1;
}
bool operator <=(const Fecha& f1, const Fecha& f2){
    return !(f2 < f1);
}
bool operator >=(const Fecha& f1, const Fecha& f2){
    return !(f1 < f2);
}
//CONSTRUCTOR DE INVALIDA----------------------------------------------
-------------------------
Fecha::Invalida::Invalida(const char* inf): info_(inf){}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·**Cadena.hpp:**

```
#ifndef _CADENA_HPP_
#define _CADENA_HPP_

#include <iostream>

class Cadena{
    public:
        //Constructores
        explicit Cadena(size_t tam = 0, char c = ' ');
        Cadena(const Cadena& cad);
        Cadena(const char* cad);
        //Operador de asignación
        Cadena& operator =(const Cadena&);
        //Conversor a const char*
        operator const char*() const;

        //Métodos Observadores
        size_t length() const noexcept;

        //Operador de Concatenación de Cadenas
        Cadena& operator +=(const Cadena& cad);


        //Operadores de Índice
        char& operator[](size_t i);
        const char& operator[](size_t i) const;
        char& at(size_t i);
        const char& at(size_t i) const;

        //Función Subcadena
        Cadena substr(size_t i, size_t tam) const;

        //Destructor
        ~Cadena();

    private:
        char* s_;
        size_t tam_;

};

//Operadores Lógicos
bool operator ==(const Cadena& cad1, const Cadena& cad2);
bool operator !=(const Cadena& cad1, const Cadena& cad2);
bool operator <(const Cadena& cad1, const Cadena& cad2);
bool operator >(const Cadena& cad1, const Cadena& cad2);
bool operator <=(const Cadena& cad1, const Cadena& cad2);
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
bool operator >=(const Cadena& cad1, const Cadena& cad2);

//Operador de concatenación de cadenas
Cadena operator +(const Cadena& cad1, const Cadena& cad2);

//Funciones Inline-------------------------------------------------------
-------------------------
inline size_t Cadena::length() const noexcept{return tam_;}

inline char& Cadena::operator[](size_t i){return s_[i];}
inline const char& Cadena::operator[](size_t i) const {return s_[i];}

inline Cadena::operator const char*() const {return s_;}

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Cadena.cpp:

```
#include "cadena.hpp"
#include <cstring>
#include <iostream>
#include <stdexcept>

//Constructores-----------------------------------------------------------
-------------------------
Cadena::Cadena(size_t tam, char c): s_{new char[tam+1]}, tam_{tam}{
    for(size_t i = 0; i < tam_; i++)
        s_[i] = c;
    s_[tam_] = '\0';
}

Cadena::Cadena(const Cadena& cad): s_{new char[cad.tam_ + 1]}, tam_{cad.t
am_}{
    strcpy(s_,cad.s_);
}

Cadena::Cadena(const char* cad): s_{new char[strlen(cad)+1]}, tam_{strlen
(cad)}{
    strcpy(s_, cad);
}

//Operador de Asgnación---------------------------------------------------
-------------------------
Cadena& Cadena::operator =(const Cadena& cad){
    if(this != &cad){ //Evitamos autoasignación
        delete[] s_;
        s_ = new char[cad.tam_ + 1];
        tam_ = cad.tam_;
        strcpy(s_, cad.s_);
    }
    return *this;
}

//Operadores de Concatenación---------------------------------------------
-------------------------
Cadena& Cadena::operator +=(const Cadena& cad){
    char* aux = new char[tam_+1];
    strcpy(aux,s_);

    tam_ += cad.tam_;

    delete[] s_;
    s_ = new char[tam_ + 1];

    strcpy(s_, aux);
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    strcat(s_, cad.s_);

    delete[] aux;
    return *this;
}

Cadena operator +(const Cadena& cad1, const Cadena& cad2){
    Cadena aux(cad1);
    return aux += cad2;

}

//Operadores Lógicos-------------------------------------------------------
-------------------------
bool operator ==(const Cadena& cad1, const Cadena& cad2){
    return !strcmp(cad1, cad2);
}

bool operator !=(const Cadena& cad1, const Cadena& cad2){
    return strcmp(cad1, cad2);
}

bool operator <(const Cadena& cad1, const Cadena& cad2){
    return (strcmp(cad1, cad2) < 0);
}

bool operator >(const Cadena& cad1, const Cadena& cad2){
    return cad2 < cad1;
}

bool operator <=(const Cadena& cad1, const Cadena& cad2){
    return !(cad2 < cad1);
}

bool operator >=(const Cadena& cad1, const Cadena& cad2){
    return !(cad1 < cad2);
}
//Funciones AT-------------------------------------------------------------
-------------------------
char& Cadena::at(size_t i){
    if(i < tam_)
        return s_[i];
    else
        throw std::out_of_range("Funcion at(): fuera de limites.");
}

const char& Cadena::at(size_t i) const{
    if(i < tam_)
        return s_[i];
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    else
        throw std::out_of_range("Funcion at(): fuera de limites.");

}

Cadena Cadena::substr(size_t i, size_t tam) const{
    //Al ser enteros sin signo, al introducir un número neg. da uno posit
ivo muy grande.
    if(i+tam > tam_ || tam > tam_ || i > tam_)
        throw std::out_of_range("Funcion substr(): fuera de limites.");
    else{
        Cadena aux(tam);
        strncpy(aux.s_, s_ + i, tam);
        aux.s_[tam] = '\0';
        return aux;
    }
}

//Destructor-------------------------------------------------------------
-------------------------
Cadena::~Cadena(){
    delete[] s_;
    tam_ = 0;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Makefile:

```
CXX = clang++
CXXFLAGS = -g -Wall -std=c++11 -pedantic
CPPFLAGS = -DP0 -I../Tests-auto -I.
VPATH = ../Tests-auto:.

EJEC = test-P0-auto test-P0-consola

all: test-P0-auto test-P0-consola

fecha.o: fecha.cpp fecha.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) fecha.cpp

cadena.o: cadena.cpp cadena.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) cadena.cpp

test-fechacadena-consola.o: test-fechacadena-
consola.cpp cadena.hpp fecha.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) test-fechacadena-consola.cpp

test-P0-consola: test-fechacadena-consola.o fecha.o cadena.o
    $(CXX) -o $@ $^

test-P0-auto: test-caso0-fecha-auto.o test-caso0-cadena-auto.o test-
auto.o cadena.o fecha.o
    $(CXX) -o  $@ $^

test-caso0-fecha-auto.o test-caso0-cadena-auto.o test-auto.o: test-caso0-
fecha-auto.cpp test-caso0-cadena-auto.cpp test-auto.cpp test-
auto.hpp fecha.hpp cadena.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) $^

clean:
    $(RM) $(EJEC) *.o *.gch core  *~
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Práctica 1:

·Fecha.hpp:

```
#ifndef _FECHA_HPP_
#define _FECHA_HPP_

#include <iostream>

class Fecha{
    public:
        //Constructores
        explicit Fecha(int d = 0, int m = 0, int a = 0); //Con 3, 2, 1 y
ningún parámetro
        Fecha(const Fecha& f) = default; //Ctor. de copia
        Fecha(const char* c); //A partir de una cadena de caracteres de f
ormato dd/mm/aaaa
        Fecha& operator = (const Fecha&) = default; //Operador asignación

        //Conversion a const *char
        const char* cadena() const;

        //Constantes
        static const int AnnoMinimo = 1902;
        static const int AnnoMaximo = 2037;

        //Clase Fecha Invalida
        class Invalida{
            public:
                Invalida(const char* inf);
                const char* por_que() const noexcept;
            private:
                const char* info_;
        };

        //Observadores
        int dia() const noexcept;
        int mes() const noexcept;
        int anno() const noexcept;
        //Operadores
        Fecha& operator ++();
        Fecha operator ++(int);
        Fecha& operator --();
        Fecha operator --(int);
        Fecha& operator +=(int i);
        Fecha& operator -=(int i);

        //Operadores Aritméticos
        Fecha operator +(int i) const;
        Fecha operator -(int i) const;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        //Destructor
        ~Fecha() = default;

    private:
        int dia_, mes_, anno_;
        void comprueba();
        int ultimo_dia() const;

};



//Operadores Lógicos
bool operator ==(const Fecha& f1, const Fecha& f2) noexcept;
bool operator !=(const Fecha& f1, const Fecha& f2) noexcept;
bool operator <(const Fecha& f1, const Fecha& f2) noexcept;
bool operator >(const Fecha& f1, const Fecha& f2) noexcept;
bool operator <=(const Fecha& f1, const Fecha& f2) noexcept;
bool operator >=(const Fecha& f1, const Fecha& f2) noexcept;

//Entrada/Salida-------------------------------------------------------
--------------------------
std::ostream& operator <<(std::ostream& os, const Fecha& f) noexcept;
std::istream& operator >>(std::istream& is, Fecha& f);

//Métodos Inline-------------------------------------------------------
--------------------------

inline int Fecha::dia() const noexcept{return dia_;}
inline int Fecha::mes() const noexcept{return mes_;}
inline int Fecha::anno() const noexcept{return anno_;}

//Métodos Inline INVALIDA----------------------------------------------
--------------------------
inline Fecha::Invalida::Invalida(const char* inf): info_(inf){}
inline const char* Fecha::Invalida::por_que() const noexcept{return info_
;}


#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Fecha.cpp:

```
#include "fecha.hpp"
#include <ctime>
#include <cstdio>
#include <iostream>
#include <locale>

//FUNCIONES PRIVADAS ----------------------------------------------------
-----------------------------
//Funcion que lanza excepciones si el dia o el año es incorrecto (ultimo_
dia() es llamada desde aqui)
void Fecha::comprueba(){
    //menor que el ultimo dia del mes de la fecha parametro implicito y m
ínimo 1
    if(dia_ > ultimo_dia() || dia_ < 1){
        Fecha::Invalida d_inval("Dia Incorrecto");
        throw d_inval;
    }
    //anno entre los annos validos
    if(anno_ < Fecha::AnnoMinimo || anno_ > Fecha::AnnoMaximo){
        Fecha::Invalida a_inval("Año Incorrecto");
        throw a_inval;
    }

}
//Función privada que, a partir de un mes, devuelve su ultimo dia o lanza
 una excepcion si es incorrecto
int Fecha::ultimo_dia() const {
    //Enero, marzo, mayo, julio, agosto, octubre y diciembre --> 31 dias
    if(mes_ == 1 || mes_ == 3 || mes_ == 5 || mes_ == 7 || mes_ == 8 || m
es_ == 10 || mes_ == 12)
        return 31;
    //Abril, junio, septiembre y noviembre --> 30 dias
    else if(mes_ == 4 || mes_ == 6 || mes_ == 9 || mes_ == 11)
        return 30;
    //Febrero --> 28 o 29 segun sea bisiesto o no
    else if(mes_ == 2){ //¿Es bisiesto?
        if(anno_ % 4 == 0 && (anno_ % 400 == 0 || anno_ % 100 != 0))
            return 29;
        else
            return 28;
    //El mes no esta en [1-12]
    }else{
        Fecha::Invalida inval("Mes Incorrecto");
        throw inval;
        return 0;
    }
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
//CONSTRUCTORES-------------------------------------------------------------
------------------------------
Fecha::Fecha(int d, int m, int a): dia_{d}, mes_{m}, anno_{a}{
    //Obtenemos fecha actual por si existen 0 en dia, mes o anno
    std::time_t tiempo_calendario = std::time(nullptr);
    std::tm* tiempo_descompuesto = std::localtime(&tiempo_calendario);
    //Cambio de 0 a día, mes o año actual
    if(anno_ == 0)
        anno_ = tiempo_descompuesto->tm_year +1900;

    if(mes_ == 0)
        mes_ = tiempo_descompuesto->tm_mon + 1;

    if (dia_ == 0)
        dia_ = tiempo_descompuesto->tm_mday;
    //Se comprueba si la fecha es valida
    comprueba();
}

Fecha::Fecha (const char* c){
    //Tomamos los valores directamente de la cadena
    if(sscanf(c,"%d/%d/%d",&dia_,&mes_,&anno_) != 3){
        //Si es distinto de 3, no hemos leído día, mes y año (Falta uno d
e ellos o hay otros caracteres distintos de '/')
        Fecha::Invalida format("Formato de la cadena incorrecto.");
        throw format;
    }
    //Tomamaos fecha actual por si existen 0 en dia, mes o anno
    std::time_t tiempo_calendario = std::time(nullptr);
    std::tm* tiempo_descompuesto = std::localtime(&tiempo_calendario);
    //Cambio de 0 a día, mes o año actual
    if(anno_ == 0)
        anno_ = tiempo_descompuesto->tm_year +1900;

    if(mes_ == 0)
        mes_ = tiempo_descompuesto->tm_mon + 1;

    if (dia_ == 0)
        dia_ = tiempo_descompuesto->tm_mday;
    //Se comprueba si la fecha es valida
    comprueba();
}


//CONVERSOR FECHA ---> CONST CHAR* ------------------------------------
-------------------------
const char* Fecha::cadena() const{
    //Idioma espannol
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    std::locale::global(std::locale("es_ES.utf8"));
    char *aux = new char[100];
    //Creamos una estructura tm e introducimos dia, mes y anno en ella
    std::tm f{};

    f.tm_mday = dia_;
    f.tm_mon = mes_ - 1;
    f.tm_year = anno_ - 1900;
    //Formatea la fecha y la devuelve
    mktime(&f);
    strftime(aux,100,"%A %e de %B de %Y",&f);
    return aux;

}

//OPERADORES-------------------------------------------------------------
-------------------------
Fecha& Fecha::operator +=(int i){//f += i
    //Creamos una estructura con la fecha ya operada con el entero y la f
ormateamos con mktime(p. ej. cambiamos de 32-4-2020 a 2-5-2020)
    std::tm f{};
    f.tm_mday = dia_ + i;
    f.tm_mon = mes_ - 1;
    f.tm_year = anno_ - 1900;
    std::mktime(&f);

    //Pasamos la fecha formateada a los atributos del parametro implicito
    dia_ = f.tm_mday;
    mes_ = f.tm_mon + 1;
    anno_ = f.tm_year + 1900;
    comprueba();
    return *this;
}

Fecha& Fecha::operator -=(int i){ //f -= i
    *this += -i;
    return *this;
}

Fecha& Fecha::operator ++(){ //++f

    *this += 1;
    return *this;
}

Fecha Fecha::operator ++(int){ //f++
    Fecha *f = new Fecha(*this);
    *this += 1;
    return *f;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
}

Fecha& Fecha::operator --(){ //--f
    *this += -1;
    return *this;
}

Fecha Fecha::operator --(int){ //f--
    Fecha *f = new Fecha(*this);
    *this += -1;
    return *f;
}

//OPERADORES ARTIMÉTICOS-----------------------------------------------
-------------------------
Fecha Fecha::operator +(int i) const{//f + i
    Fecha f(*this);
    return f += i;
}

Fecha Fecha::operator -(int i) const{//f - i
    Fecha f(*this);
    return f += -i;
}

//OPERADORES LÓGICOS---------------------------------------------------
-------------------------
bool operator ==(const Fecha& f1, const Fecha& f2) noexcept{//f1 == f2
    //Comparamos campo a campo
    return (f1.dia() == f2.dia() && f1.mes() == f2.mes() && f1.anno() ==
f2.anno());
}
bool operator !=(const Fecha& f1, const Fecha& f2) noexcept{//f1 != f2
    return !(f1 == f2);
}
bool operator <(const Fecha& f1, const Fecha& f2) noexcept{// f1 < f2
    /* Para devolver true, debe ser una de estas opciones:
    *   -Anno del primero menor que anno del segundo
    *   -Annos iguales, mes del primero menor que mes del segundo
    *   -Annos y meses iguales, dia del primero menor que dia del segundo
    */
    return (f1.anno() < f2.anno() || (f1.anno() == f2.anno() && (f1.mes()
 < f2.mes() || (f1.mes() == f2.mes() && f1.dia() < f2.dia()))));
}
bool operator >(const Fecha& f1, const Fecha& f2) noexcept{//f1 > f2
    return f2 < f1;
}
bool operator <=(const Fecha& f1, const Fecha& f2) noexcept{//f1 <= f2
    return !(f2 < f1);
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
}
bool operator >=(const Fecha& f1, const Fecha& f2) noexcept{// f1 >= f2
    return !(f1 < f2);
}

//ENTRADA/SALIDA-------------------------------------------------------
-------------------------
std::ostream& operator <<(std::ostream& os, const Fecha& f) noexcept{ //s
alida
    //Imprimimos la fecha formateada ("nombre_dia_semana, dia_num de nomb
re_mes de anno_num")
    os << f.cadena();
    return os;
}

std::istream& operator >>(std::istream& is, Fecha& f){ //entrada
    char linea[11];
    is.getline(linea, 11);
    try{
        //Llamada a ctor. de fecha a partir de una cadena, y al operador
de asig.
        f = linea;
    }catch(Fecha::Invalida e){
        //Failbit
        is.setstate(std::ios::failbit);
        throw e;
    }
    return is;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·**Cadena.hpp:**

```
#ifndef _CADENA_HPP_
#define _CADENA_HPP_

#include <iostream>

class Cadena{
    public:
        //Constructores
        explicit Cadena(size_t tam = 0, char c = ' ');
        Cadena(const Cadena& cad);
        Cadena(const char* cad);
        Cadena (Cadena&& cad);

        //Operadores de asignación
        Cadena& operator =(const Cadena& cad);
        Cadena& operator =(Cadena&& cad);

        //Conversor a const char*
        const char* c_str() const noexcept;

        //Métodos Observadores
        size_t length() const noexcept;

        //Operador de Concatenación de Cadenas
        Cadena& operator +=(const Cadena& cad);


        //Operadores de Índice
        char& operator[](size_t i);
        const char& operator[](size_t i) const;
        char& at(size_t i);
        const char& at(size_t i) const;

        //Función Subcadena
        Cadena substr(size_t i, size_t tam) const;

        //Iteradores
        typedef char* iterator;
        typedef const char* const_iterator;
        typedef std::reverse_iterator<iterator> reverse_iterator;
        typedef std::reverse_iterator<const_iterator> const_reverse_itera
tor;

        //Funciones de iteradores
        iterator begin() noexcept;
        const_iterator begin() const noexcept;
        reverse_iterator rbegin() noexcept;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        const_reverse_iterator rbegin() const noexcept;
        const_iterator cbegin() const noexcept;
        const_reverse_iterator crbegin() const noexcept;
        iterator end() noexcept;
        const_iterator end() const noexcept;
        reverse_iterator rend() noexcept;
        const_reverse_iterator rend() const noexcept;
        const_iterator cend() const noexcept;
        const_reverse_iterator crend() const noexcept;

        //Destructor
        ~Cadena();

    private:
        char* s_;
        size_t tam_;

};

//Operadores Lógicos
bool operator ==(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator !=(const Cadena& cad1, const Cadena& cad2)noexcept;
bool operator <(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator >(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator <=(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator >=(const Cadena& cad1, const Cadena& cad2) noexcept;

//Operador de concatenación de cadenas
Cadena operator +(const Cadena& cad1, const Cadena& cad2);

//Entrada/Salida
std::ostream& operator <<(std::ostream& os, const Cadena& cad) noexcept;
std::istream& operator >>(std::istream& is, Cadena& cad);
//Funciones Inline-------------------------------------------------------
-------------------------
inline size_t Cadena::length() const noexcept{
    return tam_;
}

inline char& Cadena::operator[](size_t i){
    return s_[i];
}
inline const char& Cadena::operator[](size_t i) const {
    return s_[i];
}

inline const char* Cadena::c_str() const noexcept{
    return s_;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
inline Cadena::iterator Cadena::begin() noexcept{
    return s_;
}
inline Cadena::const_iterator Cadena::begin() const noexcept{
    return const_iterator(s_);
}
inline Cadena::iterator Cadena::end() noexcept{
    return s_+tam_;
}
inline Cadena::const_iterator Cadena::end() const noexcept{
    return const_iterator(s_+tam_);
}
inline Cadena::reverse_iterator Cadena::rbegin() noexcept{
    return reverse_iterator(end());
}
inline Cadena::const_reverse_iterator Cadena::rbegin() const noexcept{
    return const_reverse_iterator(end());
}
inline Cadena::const_iterator Cadena::cbegin() const noexcept{
    return const_iterator(s_);
}
inline Cadena::const_reverse_iterator Cadena::crbegin() const noexcept{
    return const_reverse_iterator(end());
}
inline Cadena::reverse_iterator Cadena::rend() noexcept{
    return reverse_iterator(begin());
}
inline Cadena::const_reverse_iterator Cadena::rend() const noexcept{
    return const_reverse_iterator(begin());
}
inline Cadena::const_iterator Cadena::cend() const noexcept{
    return const_iterator(s_+tam_);
}
inline Cadena::const_reverse_iterator Cadena::crend() const noexcept{
    return const_reverse_iterator(begin());
}

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Cadena.cpp:

```
#include "cadena.hpp"
#include <cstring>
#include <iostream>
#include <stdexcept>

//Constructores-----------------------------------------------------------
-------------------------
Cadena::Cadena(size_t tam, char c): s_{new char[tam+1]}, tam_{tam}{ //Pre
det.
    for(size_t i = 0; i < tam_; i++)
        s_[i] = c;
    s_[tam_] = '\0';
}

Cadena::Cadena(const Cadena& cad): s_{new char[cad.tam_ + 1]}, tam_{cad.t
am_}{ //Copia
    strcpy(s_,cad.s_);
}

Cadena::Cadena(const char* cad): s_{new char[strlen(cad)+1]}, tam_{strlen
(cad)}{ //A partir de cadena bajo nivel
    strcpy(s_, cad);
}

Cadena::Cadena(Cadena&& cad): s_{cad.s_}, tam_{cad.tam_}{ //Movimiento
    cad.tam_ = 0;
    cad.s_ = nullptr;
}

//Operador de Asgnación---------------------------------------------------
-------------------------
Cadena& Cadena::operator =(const Cadena& cad){ //Op. Asig. por copia
    if(this != &cad){ //Evitamos autoasignación
        tam_ = cad.tam_;
        delete[] s_;
        s_ = new char[tam_ + 1];
        strcpy(s_, cad.s_);
    }
    return *this;
}

Cadena& Cadena::operator = (Cadena&& cad){ //Op. Asig. Semántica de Movim
iento
    if(this != &cad){
        tam_ = cad.tam_;
        delete[] s_;
        s_ = cad.s_;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        cad.s_ = nullptr;
        cad.tam_ = 0;
    }
    return *this;

}
//Operadores de Concatenación-----------------------------------------
-------------------------
Cadena& Cadena::operator +=(const Cadena& cad){
    char* aux = new char[tam_+1];
    strcpy(aux,s_);

    tam_ += cad.tam_;

    delete[] s_;
    s_ = new char[tam_ + 1];

    strcpy(s_, aux);
    strcat(s_, cad.s_);

    delete[] aux;
    return *this;
}

Cadena operator +(const Cadena& cad1, const Cadena& cad2){
    Cadena aux(cad1);
    return aux += cad2;

}

//Operadores Lógicos--------------------------------------------------
-------------------------
bool operator ==(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !strcmp(cad1.c_str(), cad2.c_str());
}

bool operator !=(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !(cad1 == cad2);
}

bool operator <(const Cadena& cad1, const Cadena& cad2) noexcept{
    //Si alfabéticamente es menor, strcmp devuelve un valor negativo
    return (strcmp(cad1.c_str(), cad2.c_str()) < 0);
}

bool operator >(const Cadena& cad1, const Cadena& cad2) noexcept{
    return cad2 < cad1;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
bool operator <=(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !(cad2 < cad1);
}

bool operator >=(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !(cad1 < cad2);
}
//Funciones AT-------------------------------------------------------------
-------------------------
char& Cadena::at(size_t i){
    if(i < tam_)
        return s_[i];
    else
        throw std::out_of_range("Funcion at(): fuera de limites.");
}

const char& Cadena::at(size_t i) const{
    if(i < tam_)
        return s_[i];
    else
        throw std::out_of_range("Funcion at(): fuera de limites.");

}

Cadena Cadena::substr(size_t i, size_t tam) const{
    //Al ser enteros sin signo, si introducimos un número neg. da uno pos
itivo muy grande (Deborda).
    if(i+tam > tam_ || tam > tam_ || i > tam_)
        throw std::out_of_range("Funcion substr(): fuera de limites.");
    else{
        Cadena aux(tam);
        strncpy(aux.s_, s_ + i, tam);
        aux.s_[tam] = '\0';
        return aux;
    }
}
//Entrada/Salida-----------------------------------------------------------
-------------------------
std::ostream& operator <<(std::ostream& os, const Cadena& cad) noexcept{
//Salida
    //Imprimimos la cadena de bajo nivel (atributo privado s_)
    os << cad.c_str();
    return os;
}
std::istream& operator >>(std::istream& is, Cadena& cad){ //Entrada
    //Creamos una cadena de bajo nivel de 33 caracteres (32 + '\0')
    char linea[33] = "";
    is.width(33); //32 caracteres + carácter nulo
    is >> linea;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    //Llamada al ctor de cadena a partir de una de bajo nivel y al operad
or de asig.
    cad = linea;
    return is;

}
//Destructor-------------------------------------------------------------
--------------------------
Cadena::~Cadena(){
    //Borramos el contenido de la cadena de bajo nivel, el tamanno se bor
raria automaticamente
    delete[] s_;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·**Makefile:**

```
CXX = clang++
CXXFLAGS = -g -Wall -std=c++11 -pedantic
CPPFLAGS = -DP1 -I../Tests-auto -I.
VPATH = ../Tests-auto:.

#Archivos ejecutables
EJEC = test-P0-auto test-P0-consola

all: test-P0-auto test-P0-consola

fecha.o: fecha.cpp fecha.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) fecha.cpp

cadena.o: cadena.cpp cadena.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) cadena.cpp

test-fechacadena-consola.o: test-fechacadena-
consola.cpp cadena.hpp fecha.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) test-fechacadena-consola.cpp

test-P0-consola: test-fechacadena-consola.o fecha.o cadena.o
    $(CXX) -o $@ $^

test-P0-auto: test-caso0-fecha-auto.o test-caso0-cadena-auto.o test-
auto.o cadena.o fecha.o
    $(CXX) -o  $@ $^

test-caso0-fecha-auto.o test-caso0-cadena-auto.o test-auto.o: test-caso0-
fecha-auto.cpp test-caso0-cadena-auto.cpp test-auto.cpp test-
auto.hpp fecha.hpp cadena.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) $^

fecha_cadena_check:
    $(MAKE) -f Make_check.mk $@
clean:
    $(RM) $(EJEC) *.o *.gch core  *~
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Practica 2:

·Fecha.hpp:

```
#ifndef _FECHA_HPP_
#define _FECHA_HPP_

#include <iostream>

class Fecha{
    public:
        //Constructores
        explicit Fecha(int d = 0, int m = 0, int a = 0); //Con 3, 2, 1 y
ningún parámetro
        Fecha(const Fecha& f) = default; //Ctor. de copia
        Fecha(const char* c); //A partir de una cadena de caracteres de f
ormato dd/mm/aaaa
        Fecha& operator = (const Fecha&) = default; //Operador asignación

        //Conversion a const *char
        const char* cadena() const;

        //Constantes
        static const int AnnoMinimo = 1902;
        static const int AnnoMaximo = 2037;

        //Clase Fecha Invalida
        class Invalida{
            public:
                Invalida(const char* inf);
                const char* por_que() const noexcept;
            private:
                const char* info_;
        };

        //Observadores
        int dia() const noexcept;
        int mes() const noexcept;
        int anno() const noexcept;
        //Operadores
        Fecha& operator ++();
        Fecha operator ++(int);
        Fecha& operator --();
        Fecha operator --(int);
        Fecha& operator +=(int i);
        Fecha& operator -=(int i);

        //Operadores Aritméticos
        Fecha operator +(int i) const;
        Fecha operator -(int i) const;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        //Destructor
        ~Fecha() = default;

    private:
        int dia_, mes_, anno_;
        void comprueba();
        int ultimo_dia() const;

};



//Operadores Lógicos
bool operator ==(const Fecha& f1, const Fecha& f2) noexcept;
bool operator !=(const Fecha& f1, const Fecha& f2) noexcept;
bool operator <(const Fecha& f1, const Fecha& f2) noexcept;
bool operator >(const Fecha& f1, const Fecha& f2) noexcept;
bool operator <=(const Fecha& f1, const Fecha& f2) noexcept;
bool operator >=(const Fecha& f1, const Fecha& f2) noexcept;

//Entrada/Salida-------------------------------------------------------
--------------------------
std::ostream& operator <<(std::ostream& os, const Fecha& f) noexcept;
std::istream& operator >>(std::istream& is, Fecha& f);

//Métodos Inline-------------------------------------------------------
--------------------------

inline int Fecha::dia() const noexcept{return dia_;}
inline int Fecha::mes() const noexcept{return mes_;}
inline int Fecha::anno() const noexcept{return anno_;}

//Métodos Inline INVALIDA----------------------------------------------
--------------------------
inline Fecha::Invalida::Invalida(const char* inf): info_(inf){}
inline const char* Fecha::Invalida::por_que() const noexcept{return info_
;}


#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Fecha.cpp:

```
#include "fecha.hpp"
#include <ctime>
#include <cstdio>
#include <iostream>
#include <locale>

//FUNCIONES PRIVADAS ----------------------------------------------------
-----------------------------
//Funcion que lanza excepciones si el dia o el año es incorrecto (ultimo_
dia() es llamada desde aqui)
void Fecha::comprueba(){
    //menor que el ultimo dia del mes de la fecha parametro implicito y m
ínimo 1
    if(dia_ > ultimo_dia() || dia_ < 1){
        Fecha::Invalida d_inval("Dia Incorrecto");
        throw d_inval;
    }
    //anno entre los annos validos
    if(anno_ < Fecha::AnnoMinimo || anno_ > Fecha::AnnoMaximo){
        Fecha::Invalida a_inval("Año Incorrecto");
        throw a_inval;
    }

}
//Función privada que, a partir de un mes, devuelve su ultimo dia o lanza
 una excepcion si es incorrecto
int Fecha::ultimo_dia() const {
    //Enero, marzo, mayo, julio, agosto, octubre y diciembre --> 31 dias
    if(mes_ == 1 || mes_ == 3 || mes_ == 5 || mes_ == 7 || mes_ == 8 || m
es_ == 10 || mes_ == 12)
        return 31;
    //Abril, junio, septiembre y noviembre --> 30 dias
    else if(mes_ == 4 || mes_ == 6 || mes_ == 9 || mes_ == 11)
        return 30;
    //Febrero --> 28 o 29 segun sea bisiesto o no
    else if(mes_ == 2){ //¿Es bisiesto?
        if(anno_ % 4 == 0 && (anno_ % 400 == 0 || anno_ % 100 != 0))
            return 29;
        else
            return 28;
    //El mes no esta en [1-12]
    }else{
        Fecha::Invalida inval("Mes Incorrecto");
        throw inval;
        return 0;
    }
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
//CONSTRUCTORES-------------------------------------------------------------
------------------------------
Fecha::Fecha(int d, int m, int a): dia_{d}, mes_{m}, anno_{a}{
    //Obtenemos fecha actual por si existen 0 en dia, mes o anno
    std::time_t tiempo_calendario = std::time(nullptr);
    std::tm* tiempo_descompuesto = std::localtime(&tiempo_calendario);
    //Cambio de 0 a día, mes o año actual
    if(anno_ == 0)
        anno_ = tiempo_descompuesto->tm_year +1900;

    if(mes_ == 0)
        mes_ = tiempo_descompuesto->tm_mon + 1;

    if (dia_ == 0)
        dia_ = tiempo_descompuesto->tm_mday;
    //Se comprueba si la fecha es valida
    comprueba();
}

Fecha::Fecha (const char* c){
    //Tomamos los valores directamente de la cadena
    if(sscanf(c,"%d/%d/%d",&dia_,&mes_,&anno_) != 3){
        //Si es distinto de 3, no hemos leído día, mes y año (Falta uno d
e ellos o hay otros caracteres distintos de '/')
        Fecha::Invalida format("Formato de la cadena incorrecto.");
        throw format;
    }
    //Tomamaos fecha actual por si existen 0 en dia, mes o anno
    std::time_t tiempo_calendario = std::time(nullptr);
    std::tm* tiempo_descompuesto = std::localtime(&tiempo_calendario);
    //Cambio de 0 a día, mes o año actual
    if(anno_ == 0)
        anno_ = tiempo_descompuesto->tm_year +1900;

    if(mes_ == 0)
        mes_ = tiempo_descompuesto->tm_mon + 1;

    if (dia_ == 0)
        dia_ = tiempo_descompuesto->tm_mday;
    //Se comprueba si la fecha es valida
    comprueba();
}


//CONVERSOR FECHA ---> CONST CHAR* ------------------------------------
-------------------------
const char* Fecha::cadena() const{
    //Idioma espannol
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    std::locale::global(std::locale("es_ES.utf8"));
    char *aux = new char[100];
    //Creamos una estructura tm e introducimos dia, mes y anno en ella
    std::tm f{};

    f.tm_mday = dia_;
    f.tm_mon = mes_ - 1;
    f.tm_year = anno_ - 1900;
    //Formatea la fecha y la devuelve
    mktime(&f);
    strftime(aux,100,"%A %e de %B de %Y",&f);
    return aux;

}

//OPERADORES-------------------------------------------------------------
-------------------------
Fecha& Fecha::operator +=(int i){//f += i
    //Creamos una estructura con la fecha ya operada con el entero y la f
ormateamos con mktime(p. ej. cambiamos de 32-4-2020 a 2-5-2020)
    std::tm f{};
    f.tm_mday = dia_ + i;
    f.tm_mon = mes_ - 1;
    f.tm_year = anno_ - 1900;
    std::mktime(&f);

    //Pasamos la fecha formateada a los atributos del parametro implicito
    dia_ = f.tm_mday;
    mes_ = f.tm_mon + 1;
    anno_ = f.tm_year + 1900;
    comprueba();
    return *this;
}

Fecha& Fecha::operator -=(int i){ //f -= i
    *this += -i;
    return *this;
}

Fecha& Fecha::operator ++(){ //++f

    *this += 1;
    return *this;
}

Fecha Fecha::operator ++(int){ //f++
    Fecha *f = new Fecha(*this);
    *this += 1;
    return *f;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
}

Fecha& Fecha::operator --(){ //--f
    *this += -1;
    return *this;
}

Fecha Fecha::operator --(int){ //f--
    Fecha *f = new Fecha(*this);
    *this += -1;
    return *f;
}

//OPERADORES ARTIMÉTICOS-----------------------------------------------
-------------------------
Fecha Fecha::operator +(int i) const{//f + i
    Fecha f(*this);
    return f += i;
}

Fecha Fecha::operator -(int i) const{//f - i
    Fecha f(*this);
    return f += -i;
}

//OPERADORES LÓGICOS---------------------------------------------------
-------------------------
bool operator ==(const Fecha& f1, const Fecha& f2) noexcept{//f1 == f2
    //Comparamos campo a campo
    return (f1.dia() == f2.dia() && f1.mes() == f2.mes() && f1.anno() ==
f2.anno());
}
bool operator !=(const Fecha& f1, const Fecha& f2) noexcept{//f1 != f2
    return !(f1 == f2);
}
bool operator <(const Fecha& f1, const Fecha& f2) noexcept{// f1 < f2
    /* Para devolver true, debe ser una de estas opciones:
    *   -Anno del primero menor que anno del segundo
    *   -Annos iguales, mes del primero menor que mes del segundo
    *   -Annos y meses iguales, dia del primero menor que dia del segundo
    */
    return (f1.anno() < f2.anno() || (f1.anno() == f2.anno() && (f1.mes()
 < f2.mes() || (f1.mes() == f2.mes() && f1.dia() < f2.dia()))));
}
bool operator >(const Fecha& f1, const Fecha& f2) noexcept{//f1 > f2
    return f2 < f1;
}
bool operator <=(const Fecha& f1, const Fecha& f2) noexcept{//f1 <= f2
    return !(f2 < f1);
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
}
bool operator >=(const Fecha& f1, const Fecha& f2) noexcept{// f1 >= f2
    return !(f1 < f2);
}

//ENTRADA/SALIDA-------------------------------------------------------
-------------------------
std::ostream& operator <<(std::ostream& os, const Fecha& f) noexcept{ //s
alida
    //Imprimimos la fecha formateada ("nombre_dia_semana, dia_num de nomb
re_mes de anno_num")
    os << f.cadena();
    return os;
}

std::istream& operator >>(std::istream& is, Fecha& f){ //entrada
    char linea[11];
    is.getline(linea, 11);
    try{
        //Llamada a ctor. de fecha a partir de una cadena, y al operador
de asig.
        f = linea;
    }catch(Fecha::Invalida e){
        //Failbit
        is.setstate(std::ios::failbit);
        throw e;
    }
    return is;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Cadena.hpp:

```
#ifndef _CADENA_HPP_
#define _CADENA_HPP_

#include <iostream>
#include <string>

class Cadena{
    public:
        //Constructores
        explicit Cadena(size_t tam = 0, char c = ' ');
        Cadena(const Cadena& cad);
        Cadena(const char* cad);
        Cadena (Cadena&& cad);

        //Operadores de asignación
        Cadena& operator =(const Cadena& cad);
        Cadena& operator =(Cadena&& cad);

        //Conversor a const char*
        const char* c_str() const noexcept;

        //Métodos Observadores
        size_t length() const noexcept;

        //Operador de Concatenación de Cadenas
        Cadena& operator +=(const Cadena& cad);


        //Operadores de Índice
        char& operator[](size_t i);
        const char& operator[](size_t i) const;
        char& at(size_t i);
        const char& at(size_t i) const;

        //Función Subcadena
        Cadena substr(size_t i, size_t tam) const;

        //Iteradores
        typedef char* iterator;
        typedef const char* const_iterator;
        typedef std::reverse_iterator<iterator> reverse_iterator;
        typedef std::reverse_iterator<const_iterator> const_reverse_itera
tor;

        //Funciones de iteradores
        iterator begin() noexcept;
        const_iterator begin() const noexcept;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        reverse_iterator rbegin() noexcept;
        const_reverse_iterator rbegin() const noexcept;
        const_iterator cbegin() const noexcept;
        const_reverse_iterator crbegin() const noexcept;
        iterator end() noexcept;
        const_iterator end() const noexcept;
        reverse_iterator rend() noexcept;
        const_reverse_iterator rend() const noexcept;
        const_iterator cend() const noexcept;
        const_reverse_iterator crend() const noexcept;

        //Destructor
        ~Cadena();

    private:
        char* s_;
        size_t tam_;

};

//Operadores Lógicos
bool operator ==(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator !=(const Cadena& cad1, const Cadena& cad2)noexcept;
bool operator <(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator >(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator <=(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator >=(const Cadena& cad1, const Cadena& cad2) noexcept;

//Operador de concatenación de cadenas
Cadena operator +(const Cadena& cad1, const Cadena& cad2);

//Entrada/Salida
std::ostream& operator <<(std::ostream& os, const Cadena& cad) noexcept;
std::istream& operator >>(std::istream& is, Cadena& cad);
//Funciones Inline-------------------------------------------------------
-------------------------
inline size_t Cadena::length() const noexcept{
    return tam_;
}

inline char& Cadena::operator[](size_t i){
    return s_[i];
}
inline const char& Cadena::operator[](size_t i) const {
    return s_[i];
}

inline const char* Cadena::c_str() const noexcept{
    return s_;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
}


inline Cadena::iterator Cadena::begin() noexcept{
    return s_;
}
inline Cadena::const_iterator Cadena::begin() const noexcept{
    return const_iterator(s_);
}
inline Cadena::iterator Cadena::end() noexcept{
    return s_+tam_;
}
inline Cadena::const_iterator Cadena::end() const noexcept{
    return const_iterator(s_+tam_);
}
inline Cadena::reverse_iterator Cadena::rbegin() noexcept{
    return reverse_iterator(end());
}
inline Cadena::const_reverse_iterator Cadena::rbegin() const noexcept{
    return const_reverse_iterator(end());
}
inline Cadena::const_iterator Cadena::cbegin() const noexcept{
    return const_iterator(s_);
}
inline Cadena::const_reverse_iterator Cadena::crbegin() const noexcept{
    return const_reverse_iterator(end());
}
inline Cadena::reverse_iterator Cadena::rend() noexcept{
    return reverse_iterator(begin());
}
inline Cadena::const_reverse_iterator Cadena::rend() const noexcept{
    return const_reverse_iterator(begin());
}
inline Cadena::const_iterator Cadena::cend() const noexcept{
    return const_iterator(s_+tam_);
}
inline Cadena::const_reverse_iterator Cadena::crend() const noexcept{
    return const_reverse_iterator(begin());
}
//Para P2 y ss
//Especializacion de la plantilla hash<T> para definir la
//funcion hash a utilizar con contenedores desordenados de
//Cadena. unordered_[st|map|multiset|multimap]
namespace std{ //Estaremos dentro del espacio de nombres std
    template <>//Es una especializacion de una plantilla de Cadena
    struct hash<Cadena>{ //Es una clase con solo un operador publico
        size_t operator() (const Cadena& cad) const //El operador funcion
        {
            hash<string> hs;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
            const char* p = cad.c_str();
            string s(p);
            size_t res = hs(s);
            return res;
        }
    };


}


#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Cadena.cpp:

```
#include "cadena.hpp"
#include <cstring>
#include <iostream>
#include <stdexcept>

//Constructores-----------------------------------------------------------
-------------------------
Cadena::Cadena(size_t tam, char c): s_{new char[tam+1]}, tam_{tam}{ //Pre
det.
    for(size_t i = 0; i < tam_; i++)
        s_[i] = c;
    s_[tam_] = '\0';
}

Cadena::Cadena(const Cadena& cad): s_{new char[cad.tam_ + 1]}, tam_{cad.t
am_}{ //Copia
    strcpy(s_,cad.s_);
}

Cadena::Cadena(const char* cad): s_{new char[strlen(cad)+1]}, tam_{strlen
(cad)}{ //A partir de cadena bajo nivel
    strcpy(s_, cad);
}

Cadena::Cadena(Cadena&& cad): s_{cad.s_}, tam_{cad.tam_}{ //Movimiento
    cad.tam_ = 0;
    cad.s_ = nullptr;
}

//Operador de Asgnación---------------------------------------------------
-------------------------
Cadena& Cadena::operator =(const Cadena& cad){ //Op. Asig. por copia
    if(this != &cad){ //Evitamos autoasignación
        tam_ = cad.tam_;
        delete[] s_;
        s_ = new char[tam_ + 1];
        strcpy(s_, cad.s_);
    }
    return *this;
}

Cadena& Cadena::operator = (Cadena&& cad){ //Op. Asig. Semántica de Movim
iento
    if(this != &cad){
        tam_ = cad.tam_;
        delete[] s_;
        s_ = cad.s_;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        cad.s_ = nullptr;
        cad.tam_ = 0;
    }
    return *this;

}
//Operadores de Concatenación-----------------------------------------
-------------------------
Cadena& Cadena::operator +=(const Cadena& cad){
    char* aux = new char[tam_+1];
    strcpy(aux,s_);

    tam_ += cad.tam_;

    delete[] s_;
    s_ = new char[tam_ + 1];

    strcpy(s_, aux);
    strcat(s_, cad.s_);

    delete[] aux;
    return *this;
}

Cadena operator +(const Cadena& cad1, const Cadena& cad2){
    Cadena aux(cad1);
    return aux += cad2;

}

//Operadores Lógicos--------------------------------------------------
-------------------------
bool operator ==(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !strcmp(cad1.c_str(), cad2.c_str());
}

bool operator !=(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !(cad1 == cad2);
}

bool operator <(const Cadena& cad1, const Cadena& cad2) noexcept{
    //Si alfabéticamente es menor, strcmp devuelve un valor negativo
    return (strcmp(cad1.c_str(), cad2.c_str()) < 0);
}

bool operator >(const Cadena& cad1, const Cadena& cad2) noexcept{
    return cad2 < cad1;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
bool operator <=(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !(cad2 < cad1);
}

bool operator >=(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !(cad1 < cad2);
}
//Funciones AT-------------------------------------------------------------
-------------------------
char& Cadena::at(size_t i){
    if(i < tam_)
        return s_[i];
    else
        throw std::out_of_range("Funcion at(): fuera de limites.");
}

const char& Cadena::at(size_t i) const{
    if(i < tam_)
        return s_[i];
    else
        throw std::out_of_range("Funcion at(): fuera de limites.");

}

Cadena Cadena::substr(size_t i, size_t tam) const{
    //Al ser enteros sin signo, si introducimos un número neg. da uno pos
itivo muy grande (Deborda).
    if(i+tam > tam_ || tam > tam_ || i > tam_)
        throw std::out_of_range("Funcion substr(): fuera de limites.");
    else{
        Cadena aux(tam);
        strncpy(aux.s_, s_ + i, tam);
        aux.s_[tam] = '\0';
        return aux;
    }
}
//Entrada/Salida-----------------------------------------------------------
-------------------------
std::ostream& operator <<(std::ostream& os, const Cadena& cad) noexcept{
//Salida
    //Imprimimos la cadena de bajo nivel (atributo privado s_)
    os << cad.c_str();
    return os;
}
std::istream& operator >>(std::istream& is, Cadena& cad){ //Entrada
    //Creamos una cadena de bajo nivel de 33 caracteres (32 + '\0')
    char linea[33] = "";
    is.width(33); //32 caracteres + carácter nulo
    is >> linea;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    //Llamada al ctor de cadena a partir de una de bajo nivel y al operad
or de asig.
    cad = linea;
    return is;

}
//Destructor-------------------------------------------------------------
--------------------------
Cadena::~Cadena(){
    //Borramos el contenido de la cadena de bajo nivel, el tamanno se bor
raria automaticamente
    delete[] s_;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Makefile (P1):

```
CXX = clang++
CXXFLAGS = -g -Wall -std=c++11 -pedantic
CPPFLAGS = -DP1 -I../Tests-auto -I.
VPATH = ../Tests-auto:.

#Archivos ejecutables
EJEC = test-P0-auto test-P0-consola

all: test-P0-auto test-P0-consola

fecha.o: fecha.cpp fecha.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) fecha.cpp

cadena.o: cadena.cpp cadena.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) cadena.cpp

test-fechacadena-consola.o: test-fechacadena-
consola.cpp cadena.hpp fecha.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) test-fechacadena-consola.cpp

test-P0-consola: test-fechacadena-consola.o fecha.o cadena.o
    $(CXX) -o $@ $^

test-P0-auto: test-caso0-fecha-auto.o test-caso0-cadena-auto.o test-
auto.o cadena.o fecha.o
    $(CXX) -o  $@ $^

test-caso0-fecha-auto.o test-caso0-cadena-auto.o test-auto.o: test-caso0-
fecha-auto.cpp test-caso0-cadena-auto.cpp test-auto.cpp test-
auto.hpp fecha.hpp cadena.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) $^

fecha_cadena_check:
    $(MAKE) -f Make_check.mk $@
clean:
    $(RM) $(EJEC) *.o *.gch core  *~
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Articulo.hpp:

```
#ifndef _ARTICULO_HPP_
#define _ARTICULO_HPP_

#include "cadena.hpp"
#include "fecha.hpp"
#include <iostream> //ostream
#include <locale>  //Para imprimir el precio con coma
#include <iomanip> //setfill, setw

//Clase Articulo-------------------------------------------------------
-------------------------
class Articulo{
    public:
        //Constructor
        Articulo(Cadena r, Cadena t, Fecha f, double p, size_t e = 0);

        //Observadores
        const Cadena& referencia() const noexcept;
        const Cadena& titulo() const noexcept;
        const Fecha& f_publi() const noexcept;
        //Observadores y sobrecargas no const
        double precio() const noexcept;
        double& precio() noexcept;
        size_t stock() const noexcept;
        size_t& stock() noexcept;



    private:
        const Cadena referencia_, titulo_;
        const Fecha f_publi_;
        double precio_;
        size_t stock_;

};
//Operador de insercion en flujo
std::ostream& operator <<(std::ostream& os, const Articulo& art) noexcept
;


//Metodos inline
inline const Cadena& Articulo::referencia() const noexcept{return referen
cia_;}
inline const Cadena& Articulo::titulo() const noexcept{return titulo_;}
inline const Fecha& Articulo::f_publi() const noexcept{return f_publi_;}
inline double Articulo::precio() const noexcept{return precio_;}
inline double& Articulo::precio() noexcept{return precio_;}
inline size_t Articulo::stock() const noexcept{return stock_;}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
inline size_t& Articulo::stock() noexcept{return stock_;}

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Articulo.cpp:

```
#include "articulo.hpp"


//Metodos Clase Articulo

//Constructor
Articulo::Articulo(Cadena r, Cadena t, Fecha f, double p, size_t e) : ref
erencia_{r}, titulo_{t}, f_publi_{f}, precio_{p}, stock_{e}{}

//Insercion en flujo
std::ostream& operator <<(std::ostream& os, const Articulo& art) noexcept
{
    //Para poder usar la coma en el precio
    std::locale::global(std::locale(""));
    os << "[" << art.referencia() << "] " << "\"" << art.titulo() << "\",
 " << art.f_publi().anno() << ". " << std::fixed << std::setprecision(2)
<< art.precio() << " €" << std::endl;
    return os;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Tarjeta.hpp:

```
#ifndef _TARJETA_HPP_
#define _TARJETA_HPP_

#include "cadena.hpp"
#include "fecha.hpp"
#include "usuario.hpp"

#include <set>      //set numeros de tarjetas
#include <iostream> //ostream
#include <cctype>   //isdigit, isspace
#include <iomanip>  //setfill, setw

//Declaración anticipada
class Usuario;

//Clase Numero-------------------------------------------------------------
--------------------------

class Numero{
    public:
        //Constructor
        Numero(const Cadena& cad);

        //Conversor a cadena de bajo nivel
        operator const char*()const noexcept;

        //Enumerado con las razones de numero incorrecto
        enum Razon{LONGITUD, DIGITOS, NO_VALIDO};

        //Clase Numero::Incorrecto
        class Incorrecto{
            public:
                Incorrecto(Numero::Razon r);
                Numero::Razon razon() const noexcept;
            private:
                Numero::Razon razon_;
        };

    private:
        Cadena numero_;
};
//Metodos Inline

//Constructor y observador clase incorrecto
inline Numero::Incorrecto::Incorrecto(Numero::Razon r):razon_{r}{}
inline Numero::Razon Numero::Incorrecto::razon() const noexcept{return ra
zon_;}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
//Conversor a const char*
inline Numero::operator const char*() const noexcept{return numero_.c_str
();}
//Operador de comparación
inline bool operator <(const Numero& n1, const Numero& n2){ return static
_cast<Cadena>(n1) < static_cast<Cadena>(n2);}

//Clase Tarjeta-------------------------------------------------------------
-------------------------
class Tarjeta{
    public:

        //Constructores (suprimimos los de copia)
        Tarjeta(const Numero& n, Usuario& u, const Fecha& f);
        Tarjeta(const Tarjeta& t) = delete;
        Tarjeta& operator=(const Tarjeta& t) = delete;
        //Destructor
        ~Tarjeta();

        //Enumerado del tipo de la tarjeta
        enum Tipo{Otro, VISA, Mastercard, Maestro, JCB, AmericanExpress};

        //Metodos Observadores
        const Numero& numero() const noexcept;
        Usuario* titular() const noexcept;
        const Fecha& caducidad() const noexcept;
        Tipo tipo() const noexcept;
        bool activa() const noexcept;
        //Metodo para activar/desactivar la tarjeta
        bool activa(bool b = true) noexcept;

        //Clases para las excepciones
        class Caducada{
            public:
                Caducada(const Fecha& f);
                const Fecha& cuando() const noexcept;
            private:
                Fecha caducada_;
        };
        class Num_duplicado{
            public:
                Num_duplicado(const Numero& n);
                const Numero& que() const noexcept;
            private:
                Numero numero_;
        };
        class Desactivada{};

    private:
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        const Numero numero_;
        Usuario* const titular_;
        const Fecha caducidad_;
        bool activa_;

        //Solo usuario podra acceder a anula_titular
        friend class Usuario;
        void anula_titular();

        //Todas las tarjetas existentes
        static std::set<Numero> tarjetas_;

};
//Inserción en flujo
std::ostream& operator <<(std::ostream& os, const Tarjeta::Tipo& tip);
std::ostream& operator <<(std::ostream& os, const Tarjeta& t) noexcept;

//Metodos Inline

//Operador comparación
inline bool operator <(const Tarjeta& t1, const Tarjeta& t2) noexcept{
    return t1.numero() < t2.numero();
}

//Observadores
inline const Numero& Tarjeta::numero() const noexcept{
    return numero_;
}
inline Usuario* Tarjeta::titular() const noexcept{
    return titular_;
}
inline const Fecha& Tarjeta::caducidad() const noexcept{
    return caducidad_;
}
inline bool Tarjeta::activa(bool b)noexcept{
    activa_ = b;
    return activa_;
}
inline bool Tarjeta::activa() const noexcept{
    return activa_;
}

//Constructores y observadores de las clases de excepción
inline Tarjeta::Caducada::Caducada(const Fecha& f): caducada_{f}{}
inline const Fecha& Tarjeta::Caducada::cuando() const noexcept{return cad
ucada_;}

inline Tarjeta::Num_duplicado::Num_duplicado(const Numero& n): numero_{n}
{}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
inline const Numero& Tarjeta::Num_duplicado::que() const noexcept{return
numero_;}

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Tarjeta.cpp:

```
#include "tarjeta.hpp"


//Declaraciones anticipadas
std::set<Numero> Tarjeta::tarjetas_;
bool luhn(const Cadena&);

//Metodos Clase Numero-----------------------------------------------------
-------------------------

//Constructor
Numero::Numero(const Cadena& cad){
    Cadena n;
    //Recorremos la cadena
    for(auto x:cad){
        //Obviamos espacios
        if(!isspace(x)){
            //Si no es un numero --> ERROR
            if(!isdigit(x)){
                Numero::Incorrecto incorr_dig(Razon::DIGITOS);
                throw incorr_dig;
            }else
                //Vamos añadiendo los digitos a una nueva cadena
                n +=Cadena(1,x); //(tam = 1, char = x);
        }
    }
    //Longitud incorrecta --> ERROR
    if (n.length() < 13 || n.length() > 19){
        Numero::Incorrecto incorr_long(Razon::LONGITUD);
        throw incorr_long;
    }else if(!luhn(n)){//Algoritmo de Luhn falla --> ERROR
        Numero::Incorrecto incorr_novalid(Razon::NO_VALIDO);
        throw incorr_novalid;
    }else
        //Copiamos la cadena sin espacios ni caracteres distintos de digi
tos en el atributo de Numero
        numero_ = n;
}

//Métodos de la Clase Tarjeta-----------------------------------------------
-------------------------

//Constructor
Tarjeta::Tarjeta(const Numero& n, Usuario& u, const Fecha& f):numero_{n},
 titular_{&u}, caducidad_{f}, activa_{true}{
    Fecha hoy;
    //Fecha de caducidad anterior a la fecha actual --> ERROR
    if(hoy  > f){
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        Tarjeta::Caducada caduc(f);
        throw caduc;
    }else if(!tarjetas_.insert(n).second){//Tarjeta ya en el sistema --
> ERROR
        Tarjeta::Num_duplicado numdup(n);
        throw numdup;
    }else //Asignamos la tarjeta al titular
        titular_->es_titular_de(*this);
}

//Destructor
Tarjeta::~Tarjeta(){
    //Si no hemos desligado la tarjeta de su titular aun, lo hacemos
    if(titular_)
        titular_->no_es_titular_de(*this);
    //Borramos la tarjeta
    tarjetas_.erase(numero_);
}

//Metodo Observador del tipo de la tarjeta
Tarjeta::Tipo Tarjeta::tipo()const noexcept{
    //Tomamos los 2 primeros caracteres
    Cadena primeros(2);
    primeros[0] = numero_[0];
    primeros[1] = numero_[1];
    //Segun sean estos dos caracteres, devolvemos un valor del enum Tipo
    if(primeros == "34" || primeros == "37")
        return Tarjeta::AmericanExpress;
    else if(primeros[0] == '3')
        return Tarjeta::JCB;
    else if(primeros[0] == '4')
        return Tarjeta::VISA;
    else if(primeros[0] == '5')
        return Tarjeta::Mastercard;
    else if(primeros[0] == '6')
        return Tarjeta::Maestro;
    else
        return Tarjeta::Otro;

}
//Metodo que solo puede llamar usuario (friend)
void Tarjeta::anula_titular(){
    //Desactivamos la tarjeta
    activa_=false;
    //La desligamos su titular
    const_cast<Usuario*&>(titular_) = nullptr;
}

//Inserción en flujo: Tipo
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
std::ostream& operator <<(std::ostream& os, const Tarjeta::Tipo& tip){
    //cadena representando el tipo segun sea el tipo de la tarjeta
    switch(tip){
        case Tarjeta::Otro: os << "Otro"; break;
        case Tarjeta::VISA: os << "VISA"; break;
        case Tarjeta::Mastercard:   os << "MasterCard"; break;
        case Tarjeta::Maestro:  os << "Maestro"; break;
        case Tarjeta::JCB:  os << "JCB"; break;
        case Tarjeta::AmericanExpress:  os << "AmericanExpress"; break;
        default: os <<"Error Tipo Tarjeta";
    }

    return os;
}

//Insercion en flujo: Tarjeta
std::ostream& operator <<(std::ostream& os, const Tarjeta& t) noexcept{
    os << " " << std::setfill('-') << std::setw(30) <<  " " << std::endl;
    os << "/                              \\" << std::endl;

    //tipo y numero
    os << "| " << std::left << std::setfill(' ') << std::setw(28) << t.ti
po() << "|" << std::endl;
    os << "| " << std::setfill(' ') << std::setw(28) << t.numero() << "|"
 << std::endl;
    //Cadena auxiliar para mostrar el nombre en mayusculas
    Cadena cad_mostrar = t.titular()->nombre() + " " + t.titular()-
>apellidos();
    for(auto& x:cad_mostrar)
        if(islower(x))
            x = toupper(x);

    os << "| "  << std::setfill(' ') << std::setw(28) << cad_mostrar << "
|" << std::endl;
    //Fecha de vencimiento formateada
    os  << std::right << "| Caduca: " << std::setfill('0') << std::setw(2
) << (t.caducidad().mes()) << "/" << std::setw(2) << (t.caducidad().anno(
)%100) << std::setfill(' ') << std::setw(16) << "|" << std::endl;

    os << "\\                              /" << std::endl;
    os << " ----------------------------" << std::endl;
    return os;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Usuario.hpp:

```
#ifndef _USUARIO_HPP_
#define _USUARIO_HPP_

#include "articulo.hpp"
#include "cadena.hpp"
#include "tarjeta.hpp"

#include <map>              //mapa de tarjetas
#include <unordered_map>    //mapa de articulos
#include <unordered_set>    //set de usuarios
#include <cstdlib>          //rand
#include <unistd.h>         //crypto
#include <cstring>          //strcmp

//Declaraciones anticipadas
class Numero;
class Tarjeta;

//Clase Clave-------------------------------------------------------------
-------------------------
class Clave{
    public:
        //Constructor
        Clave(const char* c);
        //Metodos observadores
        Cadena clave() const noexcept;
        bool verifica(const char* c) const;

        //Enum para excepciones
        enum Razon{CORTA, ERROR_CRYPT};

        //Clase de excepcion
        class Incorrecta{
            public:
                Incorrecta(const Clave::Razon r);
                Clave::Razon razon() const noexcept;
            private:
                Clave::Razon razon_;
        };
    private:
        Cadena clave_;

};

//Metodos inline

//Observador
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
inline Cadena Clave::clave() const noexcept {return clave_;}
//Metodos de la clase de excepcion
inline Clave::Incorrecta::Incorrecta(const Clave::Razon r): razon_{r}{}
inline Clave::Razon Clave::Incorrecta::razon() const noexcept{return razo
n_;}

//Clase Usuario-------------------------------------------------------
-------------------------
class Usuario{
    public:
        //Tipos: Tarjetas y Articulos
        typedef std::map<Numero, Tarjeta*> Tarjetas;
        typedef std::unordered_map<Articulo*, unsigned int> Articulos;

        //Constructores (suprimimos los de copia)
        Usuario(const Cadena& i, const Cadena& n, const Cadena& a, const
Cadena& d, const Clave& c);
        Usuario(const Usuario& u) = delete;
        Usuario& operator =(const Usuario& u) = delete;
        //Destructor
        ~Usuario();

        //Observadores
        const Cadena& id() const noexcept;
        const Cadena& nombre() const noexcept;
        const Cadena& apellidos() const noexcept;
        const Cadena& direccion() const noexcept;
        const Tarjetas& tarjetas() const noexcept;
        const Articulos& compra() const noexcept;
        size_t n_articulos() const noexcept;

        //Metodos para relacionar/borrar relacion con objetos Tarjeta
        void es_titular_de(const Tarjeta& t);
        void no_es_titular_de(const Tarjeta& t);
        //Metodo para relacionar/borrar relacion con objetos Articulo
        void compra(const Articulo& art, size_t cant = 1);

        //Clase de excepcion
        class Id_duplicado{
            public:
                Id_duplicado(const Cadena& i);
                const Cadena& idd() const noexcept;
            private:
                Cadena idd_;
        };
        //Para imprimir la clave, el operador de insercion de flujo debe
ser friend
        friend std::ostream& operator <<(std::ostream& os, const Usuario&
 u) noexcept;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    private:

        const Cadena id_, nombre_, apellidos_, direccion_;
        Clave clave_;
        Tarjetas tarjetas_;
        Articulos articulos_;

        //Set que guarda todos los usuarios
        static std::unordered_set<Cadena> usuarios_;

};
//Mostrar los articulos que tiene en el carro el usuario u
void mostrar_carro(std::ostream& os, const Usuario& u) noexcept;

//Metodos Inline
inline const Cadena& Usuario::id() const noexcept{return id_;}
inline const Cadena& Usuario::nombre() const noexcept{return nombre_;}
inline const Cadena& Usuario::apellidos() const noexcept{return apellidos
_;}
inline const Cadena& Usuario::direccion() const noexcept{return direccion
_;}
inline const Usuario::Tarjetas& Usuario::tarjetas() const noexcept{return
 tarjetas_;}
inline const Usuario::Articulos& Usuario::compra() const noexcept{return
articulos_;}
inline size_t Usuario::n_articulos() const noexcept{return articulos_.siz
e();}

//Metodos Inline de la clase de excepcion
inline Usuario::Id_duplicado::Id_duplicado(const Cadena& i):idd_(i){}
inline const Cadena& Usuario::Id_duplicado::idd() const noexcept{return i
dd_;}

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Usuario.cpp:

```
#include "usuario.hpp"


//Declaracion adelantada
std::unordered_set<Cadena> Usuario::usuarios_;

//Metodos de la Clase Clave-----------------------------------------------
-------------------------

//Constructor
Clave::Clave(const char* c){
    char* cad;
    //Cadena corta --> ERROR
    if(strlen(c) < 5) {
        Clave::Incorrecta corta(Clave::CORTA);
        throw corta;
    }
    else{
        //Caracteres disponibles para encriptar
        const char* cjto_caracteres = "./abcdefghijlmnopqrstuvwxyzABCDEFG
HIJKLMNOPQRSTUVWXYZ0123456789";
        //Creamos salt
        char salt[2];
        salt[0] = cjto_caracteres[rand() % strlen(cjto_caracteres)];
        salt[1] = cjto_caracteres[rand() % strlen(cjto_caracteres)];

        //Comprobamos que la encriptacion se haga correctamente, si no es
 asi --> ERROR
        if((cad = (crypt(c,salt))) == NULL){
            Clave::Incorrecta cryp(Clave::ERROR_CRYPT);
            throw cryp;
        }else //Asignamos al atributo la clave encriptada
            clave_ = cad;
    }


}

//Comparacion de una cadena de bajo nivel introducida con una ya cifrada
bool Clave::verifica(const char* c) const{
    return !strcmp(crypt(c, clave_.c_str()), clave_.c_str());
}

//Metodos de la clase Usuario---------------------------------------------
-------------------------

//Constructor
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
Usuario::Usuario(const Cadena& i, const Cadena& n, const Cadena& a, const
 Cadena& d, const Clave& c): id_{i}, nombre_{n}, apellidos_{a}, direccion
_{d}, clave_{c}{
    //Usuario ya en el sistema --> ERROR
    if(!usuarios_.insert(i).second){
        Usuario::Id_duplicado iddup(i);
        throw iddup;
    }
}

//Destructor
Usuario::~Usuario(){
    //Anulamos todas las tarjetas
    for(auto& kv:tarjetas_)
        kv.second->anula_titular();
    //Borramos al usuario del set de usuarios
    usuarios_.erase(id_);
}

//Vincular Usuario--Tarjeta
void Usuario::es_titular_de(const Tarjeta& t){
    //Comprobamos que la tarjeta sea del usuario
    if(t.titular() == this)
        //La añadimos a las tarjetas del usuario
        tarjetas_[t.numero()] = const_cast<Tarjeta*>(&t);
}

//Desvincular Usuario--Tarjeta
void Usuario::no_es_titular_de(const Tarjeta& t){
    //Borramos la tarjeta de lasque almacena el usuario (la tarjeta sigue
 <viva>)
    tarjetas_.erase(t.numero());
}

//Comprar articulo (Vincular/Desvincular Usuario--Articulo)
void Usuario::compra(const Articulo& art, size_t cant){
    //Buscamos el articulo en el carrito
    auto en_cesta = articulos_.find(const_cast<Articulo*>(&art));
    //No esta en el carrito --
> Añadimos el articulo si sus unidades son distintas de 0
    if(en_cesta == articulos_.end()){//Nuevo articulo
        if(cant > 0)
            articulos_[const_cast<Articulo*>(&art)] = cant;

    }else{//Articulo ya en cesta
        if(cant > 0)//Sumamos cantidades
            en_cesta->second = cant;
        else //Borramos articulo
            articulos_.erase(en_cesta);
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    }
}

//Operador de insercion en flujo
std::ostream& operator <<(std::ostream& os, const Usuario& u) noexcept{
    //datos del usuario
    os << u.id_ << " [" << u.clave_.clave() << "] "  << u.nombre_ << " "
<< u.apellidos_ << std::endl;
    os << u.direccion_ << std::endl;
    //tarjetas
    os << "Tarjetas: " << std::endl;
    for(auto& i:u.tarjetas_)
        os << *i.second << std::endl;
    return os;
}

//Mostrar los articulos que aparecen en el atributo articulos_
void mostrar_carro(std::ostream& os, const Usuario& u) noexcept{
    //Datos del usuario y numero de elementos en el carrito
    os << "Carrito de la compra de " << u.id() << " [Artículos: " << u.n_
articulos() << "]" << std::endl;
    os << "Cant.\tArt." <<std::endl;
    os << "=================================================================
=====================" << std::endl;
    //elementos y cantidades
    for(auto& i:u.compra())
        os << i.second << "\t" << *i.first << std::endl;

    os << std::endl;

}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Práctica 3:

·Fecha.hpp:

```
#ifndef _FECHA_HPP_
#define _FECHA_HPP_

#include <iostream>

class Fecha{
    public:
        //Constructores
        explicit Fecha(int d = 0, int m = 0, int a = 0); //Con 3, 2, 1 y
ningún parámetro
        Fecha(const Fecha& f) = default; //Ctor. de copia
        Fecha(const char* c); //A partir de una cadena de caracteres de f
ormato dd/mm/aaaa
        Fecha& operator = (const Fecha&) = default; //Operador asignación

        //Conversion a const *char
        const char* cadena() const;

        //Constantes
        static const int AnnoMinimo = 1902;
        static const int AnnoMaximo = 2037;

        //Clase Fecha Invalida
        class Invalida{
            public:
                Invalida(const char* inf);
                const char* por_que() const noexcept;
            private:
                const char* info_;
        };

        //Observadores
        int dia() const noexcept;
        int mes() const noexcept;
        int anno() const noexcept;
        //Operadores
        Fecha& operator ++();
        Fecha operator ++(int);
        Fecha& operator --();
        Fecha operator --(int);
        Fecha& operator +=(int i);
        Fecha& operator -=(int i);

        //Operadores Aritméticos
        Fecha operator +(int i) const;
        Fecha operator -(int i) const;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        //Destructor
        ~Fecha() = default;

    private:
        int dia_, mes_, anno_;
        void comprueba();
        int ultimo_dia() const;

};



//Operadores Lógicos
bool operator ==(const Fecha& f1, const Fecha& f2) noexcept;
bool operator !=(const Fecha& f1, const Fecha& f2) noexcept;
bool operator <(const Fecha& f1, const Fecha& f2) noexcept;
bool operator >(const Fecha& f1, const Fecha& f2) noexcept;
bool operator <=(const Fecha& f1, const Fecha& f2) noexcept;
bool operator >=(const Fecha& f1, const Fecha& f2) noexcept;

//Entrada/Salida-------------------------------------------------------
--------------------------
std::ostream& operator <<(std::ostream& os, const Fecha& f) noexcept;
std::istream& operator >>(std::istream& is, Fecha& f);

//Métodos Inline-------------------------------------------------------
--------------------------

inline int Fecha::dia() const noexcept{return dia_;}
inline int Fecha::mes() const noexcept{return mes_;}
inline int Fecha::anno() const noexcept{return anno_;}

//Métodos Inline INVALIDA----------------------------------------------
--------------------------
inline Fecha::Invalida::Invalida(const char* inf): info_(inf){}
inline const char* Fecha::Invalida::por_que() const noexcept{return info_
;}


#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Fecha.cpp:

```
#include "fecha.hpp"
#include <ctime>
#include <cstdio>
#include <iostream>
#include <locale>

//FUNCIONES PRIVADAS ----------------------------------------------------
-----------------------------
//Funcion que lanza excepciones si el dia o el año es incorrecto (ultimo_
dia() es llamada desde aqui)
void Fecha::comprueba(){
    //menor que el ultimo dia del mes de la fecha parametro implicito y m
ínimo 1
    if(dia_ > ultimo_dia() || dia_ < 1){
        Fecha::Invalida d_inval("Dia Incorrecto");
        throw d_inval;
    }
    //anno entre los annos validos
    if(anno_ < Fecha::AnnoMinimo || anno_ > Fecha::AnnoMaximo){
        Fecha::Invalida a_inval("Año Incorrecto");
        throw a_inval;
    }

}
//Función privada que, a partir de un mes, devuelve su ultimo dia o lanza
 una excepcion si es incorrecto
int Fecha::ultimo_dia() const {
    //Enero, marzo, mayo, julio, agosto, octubre y diciembre --> 31 dias
    if(mes_ == 1 || mes_ == 3 || mes_ == 5 || mes_ == 7 || mes_ == 8 || m
es_ == 10 || mes_ == 12)
        return 31;
    //Abril, junio, septiembre y noviembre --> 30 dias
    else if(mes_ == 4 || mes_ == 6 || mes_ == 9 || mes_ == 11)
        return 30;
    //Febrero --> 28 o 29 segun sea bisiesto o no
    else if(mes_ == 2){ //¿Es bisiesto?
        if(anno_ % 4 == 0 && (anno_ % 400 == 0 || anno_ % 100 != 0))
            return 29;
        else
            return 28;
    //El mes no esta en [1-12]
    }else{
        Fecha::Invalida inval("Mes Incorrecto");
        throw inval;
        return 0;
    }
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
//CONSTRUCTORES-------------------------------------------------------
------------------------------
Fecha::Fecha(int d, int m, int a): dia_{d}, mes_{m}, anno_{a}{
    //Obtenemos fecha actual por si existen 0 en dia, mes o anno
    std::time_t tiempo_calendario = std::time(nullptr);
    std::tm* tiempo_descompuesto = std::localtime(&tiempo_calendario);
    //Cambio de 0 a día, mes o año actual
    if(anno_ == 0)
        anno_ = tiempo_descompuesto->tm_year +1900;

    if(mes_ == 0)
        mes_ = tiempo_descompuesto->tm_mon + 1;

    if (dia_ == 0)
        dia_ = tiempo_descompuesto->tm_mday;
    //Se comprueba si la fecha es valida
    comprueba();
}

Fecha::Fecha (const char* c){
    //Tomamos los valores directamente de la cadena
    if(sscanf(c,"%d/%d/%d",&dia_,&mes_,&anno_) != 3){
        //Si es distinto de 3, no hemos leído día, mes y año (Falta uno d
e ellos o hay otros caracteres distintos de '/')
        Fecha::Invalida format("Formato de la cadena incorrecto.");
        throw format;
    }
    //Tomamaos fecha actual por si existen 0 en dia, mes o anno
    std::time_t tiempo_calendario = std::time(nullptr);
    std::tm* tiempo_descompuesto = std::localtime(&tiempo_calendario);
    //Cambio de 0 a día, mes o año actual
    if(anno_ == 0)
        anno_ = tiempo_descompuesto->tm_year +1900;

    if(mes_ == 0)
        mes_ = tiempo_descompuesto->tm_mon + 1;

    if (dia_ == 0)
        dia_ = tiempo_descompuesto->tm_mday;
    //Se comprueba si la fecha es valida
    comprueba();
}


//CONVERSOR FECHA ---> CONST CHAR* ------------------------------------
-------------------------
const char* Fecha::cadena() const{
    //Idioma espannol
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    std::locale::global(std::locale("es_ES.utf8"));
    char *aux = new char[100];
    //Creamos una estructura tm e introducimos dia, mes y anno en ella
    std::tm f{};

    f.tm_mday = dia_;
    f.tm_mon = mes_ - 1;
    f.tm_year = anno_ - 1900;
    //Formatea la fecha y la devuelve
    mktime(&f);
    strftime(aux,100,"%A %e de %B de %Y",&f);
    return aux;

}

//OPERADORES-------------------------------------------------------------
-------------------------
Fecha& Fecha::operator +=(int i){//f += i
    //Creamos una estructura con la fecha ya operada con el entero y la f
ormateamos con mktime(p. ej. cambiamos de 32-4-2020 a 2-5-2020)
    std::tm f{};
    f.tm_mday = dia_ + i;
    f.tm_mon = mes_ - 1;
    f.tm_year = anno_ - 1900;
    std::mktime(&f);

    //Pasamos la fecha formateada a los atributos del parametro implicito
    dia_ = f.tm_mday;
    mes_ = f.tm_mon + 1;
    anno_ = f.tm_year + 1900;
    comprueba();
    return *this;
}

Fecha& Fecha::operator -=(int i){ //f -= i
    *this += -i;
    return *this;
}

Fecha& Fecha::operator ++(){ //++f

    *this += 1;
    return *this;
}

Fecha Fecha::operator ++(int){ //f++
    Fecha *f = new Fecha(*this);
    *this += 1;
    return *f;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
}

Fecha& Fecha::operator --(){ //--f
    *this += -1;
    return *this;
}

Fecha Fecha::operator --(int){ //f--
    Fecha *f = new Fecha(*this);
    *this += -1;
    return *f;
}

//OPERADORES ARTIMÉTICOS-----------------------------------------------
-------------------------
Fecha Fecha::operator +(int i) const{//f + i
    Fecha f(*this);
    return f += i;
}

Fecha Fecha::operator -(int i) const{//f - i
    Fecha f(*this);
    return f += -i;
}

//OPERADORES LÓGICOS---------------------------------------------------
-------------------------
bool operator ==(const Fecha& f1, const Fecha& f2) noexcept{//f1 == f2
    //Comparamos campo a campo
    return (f1.dia() == f2.dia() && f1.mes() == f2.mes() && f1.anno() ==
f2.anno());
}
bool operator !=(const Fecha& f1, const Fecha& f2) noexcept{//f1 != f2
    return !(f1 == f2);
}
bool operator <(const Fecha& f1, const Fecha& f2) noexcept{// f1 < f2
    /* Para devolver true, debe ser una de estas opciones:
    *   -Anno del primero menor que anno del segundo
    *   -Annos iguales, mes del primero menor que mes del segundo
    *   -Annos y meses iguales, dia del primero menor que dia del segundo
    */
    return (f1.anno() < f2.anno() || (f1.anno() == f2.anno() && (f1.mes()
 < f2.mes() || (f1.mes() == f2.mes() && f1.dia() < f2.dia()))));
}
bool operator >(const Fecha& f1, const Fecha& f2) noexcept{//f1 > f2
    return f2 < f1;
}
bool operator <=(const Fecha& f1, const Fecha& f2) noexcept{//f1 <= f2
    return !(f2 < f1);
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
}
bool operator >=(const Fecha& f1, const Fecha& f2) noexcept{// f1 >= f2
    return !(f1 < f2);
}

//ENTRADA/SALIDA-------------------------------------------------------
-------------------------
std::ostream& operator <<(std::ostream& os, const Fecha& f) noexcept{ //s
alida
    //Imprimimos la fecha formateada ("nombre_dia_semana, dia_num de nomb
re_mes de anno_num")
    os << f.cadena();
    return os;
}

std::istream& operator >>(std::istream& is, Fecha& f){ //entrada
    char linea[11];
    is.getline(linea, 11);
    try{
        //Llamada a ctor. de fecha a partir de una cadena, y al operador
de asig.
        f = linea;
    }catch(Fecha::Invalida e){
        //Failbit
        is.setstate(std::ios::failbit);
        throw e;
    }
    return is;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Cadena.hpp:

```
#ifndef _CADENA_HPP_
#define _CADENA_HPP_

#include <iostream>
#include <string>

class Cadena{
    public:
        //Constructores
        explicit Cadena(size_t tam = 0, char c = ' ');
        Cadena(const Cadena& cad);
        Cadena(const char* cad);
        Cadena (Cadena&& cad);

        //Operadores de asignación
        Cadena& operator =(const Cadena& cad);
        Cadena& operator =(Cadena&& cad);

        //Conversor a const char*
        const char* c_str() const noexcept;

        //Métodos Observadores
        size_t length() const noexcept;

        //Operador de Concatenación de Cadenas
        Cadena& operator +=(const Cadena& cad);


        //Operadores de Índice
        char& operator[](size_t i);
        const char& operator[](size_t i) const;
        char& at(size_t i);
        const char& at(size_t i) const;

        //Función Subcadena
        Cadena substr(size_t i, size_t tam) const;

        //Iteradores
        typedef char* iterator;
        typedef const char* const_iterator;
        typedef std::reverse_iterator<iterator> reverse_iterator;
        typedef std::reverse_iterator<const_iterator> const_reverse_itera
tor;

        //Funciones de iteradores
        iterator begin() noexcept;
        const_iterator begin() const noexcept;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        reverse_iterator rbegin() noexcept;
        const_reverse_iterator rbegin() const noexcept;
        const_iterator cbegin() const noexcept;
        const_reverse_iterator crbegin() const noexcept;
        iterator end() noexcept;
        const_iterator end() const noexcept;
        reverse_iterator rend() noexcept;
        const_reverse_iterator rend() const noexcept;
        const_iterator cend() const noexcept;
        const_reverse_iterator crend() const noexcept;

        //Destructor
        ~Cadena();

    private:
        char* s_;
        size_t tam_;

};

//Operadores Lógicos
bool operator ==(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator !=(const Cadena& cad1, const Cadena& cad2)noexcept;
bool operator <(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator >(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator <=(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator >=(const Cadena& cad1, const Cadena& cad2) noexcept;

//Operador de concatenación de cadenas
Cadena operator +(const Cadena& cad1, const Cadena& cad2);

//Entrada/Salida
std::ostream& operator <<(std::ostream& os, const Cadena& cad) noexcept;
std::istream& operator >>(std::istream& is, Cadena& cad);
//Funciones Inline-------------------------------------------------------
-------------------------
inline size_t Cadena::length() const noexcept{
    return tam_;
}

inline char& Cadena::operator[](size_t i){
    return s_[i];
}
inline const char& Cadena::operator[](size_t i) const {
    return s_[i];
}

inline const char* Cadena::c_str() const noexcept{
    return s_;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
}


inline Cadena::iterator Cadena::begin() noexcept{
    return s_;
}
inline Cadena::const_iterator Cadena::begin() const noexcept{
    return const_iterator(s_);
}
inline Cadena::iterator Cadena::end() noexcept{
    return s_+tam_;
}
inline Cadena::const_iterator Cadena::end() const noexcept{
    return const_iterator(s_+tam_);
}
inline Cadena::reverse_iterator Cadena::rbegin() noexcept{
    return reverse_iterator(end());
}
inline Cadena::const_reverse_iterator Cadena::rbegin() const noexcept{
    return const_reverse_iterator(end());
}
inline Cadena::const_iterator Cadena::cbegin() const noexcept{
    return const_iterator(s_);
}
inline Cadena::const_reverse_iterator Cadena::crbegin() const noexcept{
    return const_reverse_iterator(end());
}
inline Cadena::reverse_iterator Cadena::rend() noexcept{
    return reverse_iterator(begin());
}
inline Cadena::const_reverse_iterator Cadena::rend() const noexcept{
    return const_reverse_iterator(begin());
}
inline Cadena::const_iterator Cadena::cend() const noexcept{
    return const_iterator(s_+tam_);
}
inline Cadena::const_reverse_iterator Cadena::crend() const noexcept{
    return const_reverse_iterator(begin());
}
//Para P2 y ss
//Especializacion de la plantilla hash<T> para definir la
//funcion hash a utilizar con contenedores desordenados de
//Cadena. unordered_[st|map|multiset|multimap]
namespace std{ //Estaremos dentro del espacio de nombres std
    template <>//Es una especializacion de una plantilla de Cadena
    struct hash<Cadena>{ //Es una clase con solo un operador publico
        size_t operator() (const Cadena& cad) const //El operador funcion
        {
            hash<string> hs;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
            const char* p = cad.c_str();
            string s(p);
            size_t res = hs(s);
            return res;
        }
    };


}


#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Cadena.cpp:

```
#include "cadena.hpp"
#include <cstring>
#include <iostream>
#include <stdexcept>

//Constructores-----------------------------------------------------------
-------------------------
Cadena::Cadena(size_t tam, char c): s_{new char[tam+1]}, tam_{tam}{ //Pre
det.
    for(size_t i = 0; i < tam_; i++)
        s_[i] = c;
    s_[tam_] = '\0';
}

Cadena::Cadena(const Cadena& cad): s_{new char[cad.tam_ + 1]}, tam_{cad.t
am_}{ //Copia
    strcpy(s_,cad.s_);
}

Cadena::Cadena(const char* cad): s_{new char[strlen(cad)+1]}, tam_{strlen
(cad)}{ //A partir de cadena bajo nivel
    strcpy(s_, cad);
}

Cadena::Cadena(Cadena&& cad): s_{cad.s_}, tam_{cad.tam_}{ //Movimiento
    cad.tam_ = 0;
    cad.s_ = nullptr;
}

//Operador de Asgnación---------------------------------------------------
-------------------------
Cadena& Cadena::operator =(const Cadena& cad){ //Op. Asig. por copia
    if(this != &cad){ //Evitamos autoasignación
        tam_ = cad.tam_;
        delete[] s_;
        s_ = new char[tam_ + 1];
        strcpy(s_, cad.s_);
    }
    return *this;
}

Cadena& Cadena::operator = (Cadena&& cad){ //Op. Asig. Semántica de Movim
iento
    if(this != &cad){
        tam_ = cad.tam_;
        delete[] s_;
        s_ = cad.s_;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        cad.s_ = nullptr;
        cad.tam_ = 0;
    }
    return *this;

}
//Operadores de Concatenación-----------------------------------------
-------------------------
Cadena& Cadena::operator +=(const Cadena& cad){
    char* aux = new char[tam_+1];
    strcpy(aux,s_);

    tam_ += cad.tam_;

    delete[] s_;
    s_ = new char[tam_ + 1];

    strcpy(s_, aux);
    strcat(s_, cad.s_);

    delete[] aux;
    return *this;
}

Cadena operator +(const Cadena& cad1, const Cadena& cad2){
    Cadena aux(cad1);
    return aux += cad2;

}

//Operadores Lógicos--------------------------------------------------
-------------------------
bool operator ==(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !strcmp(cad1.c_str(), cad2.c_str());
}

bool operator !=(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !(cad1 == cad2);
}

bool operator <(const Cadena& cad1, const Cadena& cad2) noexcept{
    //Si alfabéticamente es menor, strcmp devuelve un valor negativo
    return (strcmp(cad1.c_str(), cad2.c_str()) < 0);
}

bool operator >(const Cadena& cad1, const Cadena& cad2) noexcept{
    return cad2 < cad1;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
bool operator <=(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !(cad2 < cad1);
}

bool operator >=(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !(cad1 < cad2);
}
//Funciones AT-------------------------------------------------------------
-------------------------
char& Cadena::at(size_t i){
    if(i < tam_)
        return s_[i];
    else
        throw std::out_of_range("Funcion at(): fuera de limites.");
}

const char& Cadena::at(size_t i) const{
    if(i < tam_)
        return s_[i];
    else
        throw std::out_of_range("Funcion at(): fuera de limites.");

}

Cadena Cadena::substr(size_t i, size_t tam) const{
    //Al ser enteros sin signo, si introducimos un número neg. da uno pos
itivo muy grande (Deborda).
    if(i+tam > tam_ || tam > tam_ || i > tam_)
        throw std::out_of_range("Funcion substr(): fuera de limites.");
    else{
        Cadena aux(tam);
        strncpy(aux.s_, s_ + i, tam);
        aux.s_[tam] = '\0';
        return aux;
    }
}
//Entrada/Salida-----------------------------------------------------------
-------------------------
std::ostream& operator <<(std::ostream& os, const Cadena& cad) noexcept{
//Salida
    //Imprimimos la cadena de bajo nivel (atributo privado s_)
    os << cad.c_str();
    return os;
}
std::istream& operator >>(std::istream& is, Cadena& cad){ //Entrada
    //Creamos una cadena de bajo nivel de 33 caracteres (32 + '\0')
    char linea[33] = "";
    is.width(33); //32 caracteres + carácter nulo
    is >> linea;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    //Llamada al ctor de cadena a partir de una de bajo nivel y al operad
or de asig.
    cad = linea;
    return is;

}
//Destructor-------------------------------------------------------------
--------------------------
Cadena::~Cadena(){
    //Borramos el contenido de la cadena de bajo nivel, el tamanno se bor
raria automaticamente
    delete[] s_;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Makefile (P1):

```
CXX = clang++
CXXFLAGS = -g -Wall -std=c++11 -pedantic
CPPFLAGS = -DP1 -I../Tests-auto -I.
VPATH = ../Tests-auto:.

#Archivos ejecutables
EJEC = test-P0-auto test-P0-consola

all: test-P0-auto test-P0-consola

fecha.o: fecha.cpp fecha.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) fecha.cpp

cadena.o: cadena.cpp cadena.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) cadena.cpp

test-fechacadena-consola.o: test-fechacadena-
consola.cpp cadena.hpp fecha.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) test-fechacadena-consola.cpp

test-P0-consola: test-fechacadena-consola.o fecha.o cadena.o
    $(CXX) -o $@ $^

test-P0-auto: test-caso0-fecha-auto.o test-caso0-cadena-auto.o test-
auto.o cadena.o fecha.o
    $(CXX) -o  $@ $^

test-caso0-fecha-auto.o test-caso0-cadena-auto.o test-auto.o: test-caso0-
fecha-auto.cpp test-caso0-cadena-auto.cpp test-auto.cpp test-
auto.hpp fecha.hpp cadena.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) $^

fecha_cadena_check:
    $(MAKE) -f Make_check.mk $@
clean:
    $(RM) $(EJEC) *.o *.gch core  *~
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Articulo.hpp:

```
#ifndef _ARTICULO_HPP_
#define _ARTICULO_HPP_

#include "cadena.hpp"
#include "fecha.hpp"
#include <iostream> //ostream
#include <locale>  //Para imprimir el precio con coma
#include <iomanip> //setfill, setw

//Clase Articulo-------------------------------------------------------
-------------------------
class Articulo{
    public:
        //Constructor
        Articulo(Cadena r, Cadena t, Fecha f, double p, size_t e = 0);

        //Observadores
        const Cadena& referencia() const noexcept;
        const Cadena& titulo() const noexcept;
        const Fecha& f_publi() const noexcept;
        //Observadores y sobrecargas no const
        double precio() const noexcept;
        double& precio() noexcept;
        size_t stock() const noexcept;
        size_t& stock() noexcept;



    private:
        const Cadena referencia_, titulo_;
        const Fecha f_publi_;
        double precio_;
        size_t stock_;

};
//Operador de insercion en flujo
std::ostream& operator <<(std::ostream& os, const Articulo& art) noexcept
;


//Metodos inline
inline const Cadena& Articulo::referencia() const noexcept{return referen
cia_;}
inline const Cadena& Articulo::titulo() const noexcept{return titulo_;}
inline const Fecha& Articulo::f_publi() const noexcept{return f_publi_;}
inline double Articulo::precio() const noexcept{return precio_;}
inline double& Articulo::precio() noexcept{return precio_;}
inline size_t Articulo::stock() const noexcept{return stock_;}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
inline size_t& Articulo::stock() noexcept{return stock_;}

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Articulo.cpp:

```
#include "articulo.hpp"


//Metodos Clase Articulo

//Constructor
Articulo::Articulo(Cadena r, Cadena t, Fecha f, double p, size_t e) : ref
erencia_{r}, titulo_{t}, f_publi_{f}, precio_{p}, stock_{e}{}

//Insercion en flujo
std::ostream& operator <<(std::ostream& os, const Articulo& art) noexcept
{
    //Para poder usar la coma en el precio
    std::locale::global(std::locale(""));
    os << "[" << art.referencia() << "] " << "\"" << art.titulo() << "\",
 " << art.f_publi().anno() << ". " << std::fixed << std::setprecision(2)
<< art.precio() << " €" << std::endl;
    return os;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Tarjeta.hpp:

```
#ifndef _TARJETA_HPP_
#define _TARJETA_HPP_

#include "cadena.hpp"
#include "fecha.hpp"
#include "usuario.hpp"

#include <set>       //set numeros de tarjetas
#include <iostream>  //ostream
#include <cctype>    //isdigit, isspace
#include <iomanip>   //setfill, setw
#include <algorithm> //remove_if, find_if

//Declaración anticipada
class Usuario;

//Clase Numero-------------------------------------------------------------
-------------------------

class Numero{
    public:
        //Constructor
        Numero(const Cadena& cad);

        //Conversor a cadena de bajo nivel
        operator const char*()const noexcept;

        //Enumerado con las razones de numero incorrecto
        enum Razon{LONGITUD, DIGITOS, NO_VALIDO};

        //Clase Numero::Incorrecto
        class Incorrecto{
            public:
                Incorrecto(Numero::Razon r);
                Numero::Razon razon() const noexcept;
            private:
                Numero::Razon razon_;
        };

    private:
        Cadena numero_;
};
//Metodos Inline

//Constructor y observador clase incorrecto
inline Numero::Incorrecto::Incorrecto(Numero::Razon r):razon_{r}{}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
inline Numero::Razon Numero::Incorrecto::razon() const noexcept{return ra
zon_;}
//Conversor a const char*
inline Numero::operator const char*() const noexcept{return numero_.c_str
();}
//Operador de comparación
inline bool operator <(const Numero& n1, const Numero& n2){ return static
_cast<Cadena>(n1) < static_cast<Cadena>(n2);}

//Clase Tarjeta-------------------------------------------------------
-------------------------
class Tarjeta{
    public:

        //Constructores (suprimimos los de copia)
        Tarjeta(const Numero& n, Usuario& u, const Fecha& f);
        Tarjeta(const Tarjeta& t) = delete;
        Tarjeta& operator=(const Tarjeta& t) = delete;
        //Destructor
        ~Tarjeta();

        //Enumerado del tipo de la tarjeta
        enum Tipo{Otro, VISA, Mastercard, Maestro, JCB, AmericanExpress};

        //Metodos Observadores
        const Numero& numero() const noexcept;
        const Usuario* titular() const noexcept;
        const Fecha& caducidad() const noexcept;
        Tipo tipo() const noexcept;
        bool activa() const noexcept;
        //Metodo para activar/desactivar la tarjeta
        bool activa(bool b = true) noexcept;

        //Clases para las excepciones
        class Caducada{
            public:
                Caducada(const Fecha& f);
                const Fecha& cuando() const noexcept;
            private:
                Fecha caducada_;
        };
        class Num_duplicado{
            public:
                Num_duplicado(const Numero& n);
                const Numero& que() const noexcept;
            private:
                Numero numero_;
        };
        class Desactivada{};
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    private:
        const Numero numero_;
        Usuario* const titular_;
        const Fecha caducidad_;
        bool activa_;

        //Solo usuario podra acceder a anula_titular
        friend class Usuario;
        void anula_titular();

        //Todas las tarjetas existentes
        static std::set<Numero> tarjetas_;

};
//Inserción en flujo
std::ostream& operator <<(std::ostream& os, const Tarjeta::Tipo& tip);
std::ostream& operator <<(std::ostream& os, const Tarjeta& t) noexcept;

//Metodos Inline

//Operador comparación
inline bool operator <(const Tarjeta& t1, const Tarjeta& t2) noexcept{
    return t1.numero() < t2.numero();
}

//Observadores
inline const Numero& Tarjeta::numero() const noexcept{
    return numero_;
}
inline const Usuario* Tarjeta::titular() const noexcept{
    return titular_;
}
inline const Fecha& Tarjeta::caducidad() const noexcept{
    return caducidad_;
}
inline bool Tarjeta::activa(bool b)noexcept{
    activa_ = b;
    return activa_;
}
inline bool Tarjeta::activa() const noexcept{
    return activa_;
}

//Constructores y observadores de las clases de excepción
inline Tarjeta::Caducada::Caducada(const Fecha& f): caducada_{f}{}
inline const Fecha& Tarjeta::Caducada::cuando() const noexcept{return cad
ucada_;}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
inline Tarjeta::Num_duplicado::Num_duplicado(const Numero& n): numero_{n}
{}
inline const Numero& Tarjeta::Num_duplicado::que() const noexcept{return
numero_;}

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Tarjeta.cpp:

```
#include "tarjeta.hpp"


//Declaraciones anticipadas
std::set<Numero> Tarjeta::tarjetas_;
bool luhn(const Cadena&);

//Metodos Clase Numero-----------------------------------------------------
-------------------------
struct EsDigito{
    bool operator()(char c)const {return isdigit(c);}
    typedef char argument_type;
};

//Constructor
Numero::Numero(const Cadena& cad){
    Cadena n = cad;
    //Suprimimos espacios
    auto it1 = std::remove_if(n.begin(), n.end(), [](char x){return std::
isspace(x);});
    if(it1 != n.end()){
        int cont = 0;
        for(auto j = n.begin(); j != it1; j++, cont++);

        n = n.substr(0,cont);
    }
    //Comprobamos que sean dígitos
    auto it2 = std::find_if(n.begin(), n.end(), std::not1(EsDigito()));
    if(it2 != n.end()){
        Numero::Incorrecto incorr_dig(Razon::DIGITOS);
        throw incorr_dig;
    }
    //Longitud incorrecta --> ERROR
    if (n.length() < 13 || n.length() > 19){
        Numero::Incorrecto incorr_long(Razon::LONGITUD);
        throw incorr_long;
    }else if(!luhn(n)){//Algoritmo de Luhn falla --> ERROR
        Numero::Incorrecto incorr_novalid(Razon::NO_VALIDO);
        throw incorr_novalid;
    }else
        //Copiamos la cadena sin espacios ni caracteres distintos de digi
tos en el atributo de Numero
        numero_ = n;
}

//Métodos de la Clase Tarjeta----------------------------------------------
-------------------------
//Constructor
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
Tarjeta::Tarjeta(const Numero& n, Usuario& u, const Fecha& f):numero_{n},
 titular_{&u}, caducidad_{f}, activa_{true}{
    Fecha hoy;
    //Fecha de caducidad anterior a la fecha actual --> ERROR
    if(hoy  > f){
        Tarjeta::Caducada caduc(f);
        throw caduc;
    }else if(!tarjetas_.insert(n).second){//Tarjeta ya en el sistema --
> ERROR
        Tarjeta::Num_duplicado numdup(n);
        throw numdup;
    }else //Asignamos la tarjeta al titular
        titular_->es_titular_de(*this);
}

//Destructor
Tarjeta::~Tarjeta(){
    //Si no hemos desligado la tarjeta de su titular aun, lo hacemos
    if(titular_)
        titular_->no_es_titular_de(*this);
    //Borramos la tarjeta
    tarjetas_.erase(numero_);
}

//Metodo Observador del tipo de la tarjeta
Tarjeta::Tipo Tarjeta::tipo()const noexcept{
    //Tomamos los 2 primeros caracteres
    Cadena primeros(2);
    primeros[0] = numero_[0];
    primeros[1] = numero_[1];
    //Segun sean estos dos caracteres, devolvemos un valor del enum Tipo
    if(primeros == "34" || primeros == "37")
        return Tarjeta::AmericanExpress;
    else if(primeros[0] == '3')
        return Tarjeta::JCB;
    else if(primeros[0] == '4')
        return Tarjeta::VISA;
    else if(primeros[0] == '5')
        return Tarjeta::Mastercard;
    else if(primeros[0] == '6')
        return Tarjeta::Maestro;
    else
        return Tarjeta::Otro;

}
//Metodo que solo puede llamar usuario (friend)
void Tarjeta::anula_titular(){
    //Desactivamos la tarjeta
    activa_=false;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    //La desligamos su titular
    (Usuario*&)titular_ = nullptr;
}

//Insercíon en flujo: Tipo
std::ostream& operator <<(std::ostream& os, const Tarjeta::Tipo& tip){
    //cadena representando el tipo segun sea el tipo de la tarjeta
    switch(tip){
        case Tarjeta::Otro: os << "Tipo indeterminado"; break;
        case Tarjeta::VISA: os << "VISA"; break;
        case Tarjeta::Mastercard:   os << "MasterCard"; break;
        case Tarjeta::Maestro:  os << "Maestro"; break;
        case Tarjeta::JCB:  os << "JCB"; break;
        case Tarjeta::AmericanExpress:  os << "AmericanExpress"; break;
        default: os <<"Error Tipo Tarjeta";
    }

    return os;
}

//Insercion en flujo: Tarjeta
std::ostream& operator <<(std::ostream& os, const Tarjeta& t) noexcept{
    os << " " << std::setfill('-') << std::setw(30) <<  " " << std::endl;
    os << "/                              \\" << std::endl;

    //tipo y numero
    os << "| " << std::left << std::setfill(' ') << std::setw(28) << t.ti
po() << "|" << std::endl;
    os << "| " << std::setfill(' ') << std::setw(28) << t.numero() << "|"
 << std::endl;
    //Cadena auxiliar para mostrar el nombre en mayusculas
    Cadena cad_mostrar = t.titular()->nombre() + " " + t.titular()-
>apellidos();
    for(auto& x:cad_mostrar)
        if(islower(x))
            x = toupper(x);

    os << "| "  << std::setfill(' ') << std::setw(28) << cad_mostrar << "
|" << std::endl;
    //Fecha de vencimiento formateada
    os  << std::right << "| Caduca: " << std::setfill('0') << std::setw(2
) << (t.caducidad().mes()) << "/" << std::setw(2) << (t.caducidad().anno(
)%100) << std::setfill(' ') << std::setw(16) << "|" << std::endl;

    os << "\\                              /" << std::endl;
    os << " ----------------------------" << std::endl;
    return os;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Usuario.hpp:

```
#ifndef _USUARIO_HPP_
#define _USUARIO_HPP_

#include "articulo.hpp"
#include "cadena.hpp"
#include "tarjeta.hpp"

#include <map>              //mapa de tarjetas
#include <unordered_map>    //mapa de articulos
#include <unordered_set>    //set de usuarios
#include <cstdlib>          //rand
#include <unistd.h>         //crypto
#include <cstring>          //strcmp

//Declaraciones anticipadas
class Numero;
class Tarjeta;

//Clase Clave-------------------------------------------------------------
-------------------------
class Clave{
    public:
        //Constructor
        Clave(const char* c);
        //Metodos observadores
        Cadena clave() const noexcept;
        bool verifica(const char* c) const;

        //Enum para excepciones
        enum Razon{CORTA, ERROR_CRYPT};

        //Clase de excepcion
        class Incorrecta{
            public:
                Incorrecta(const Clave::Razon r);
                Clave::Razon razon() const noexcept;
            private:
                Clave::Razon razon_;
        };
    private:
        Cadena clave_;

};

//Metodos inline

//Observador
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
inline Cadena Clave::clave() const noexcept {return clave_;}
//Metodos de la clase de excepcion
inline Clave::Incorrecta::Incorrecta(const Clave::Razon r): razon_{r}{}
inline Clave::Razon Clave::Incorrecta::razon() const noexcept{return razo
n_;}

//Clase Usuario-------------------------------------------------------
-------------------------
class Usuario{
    public:
        //Tipos: Tarjetas y Articulos
        typedef std::map<Numero, Tarjeta*> Tarjetas;
        typedef std::unordered_map<Articulo*, unsigned int> Articulos;

        //Constructores (suprimimos los de copia)
        Usuario(const Cadena& i, const Cadena& n, const Cadena& a, const
Cadena& d, const Clave& c);
        Usuario(const Usuario& u) = delete;
        Usuario& operator =(const Usuario& u) = delete;
        //Destructor
        ~Usuario();

        //Observadores
        const Cadena& id() const noexcept;
        const Cadena& nombre() const noexcept;
        const Cadena& apellidos() const noexcept;
        const Cadena& direccion() const noexcept;
        const Tarjetas& tarjetas() const noexcept;
        const Articulos& compra() const noexcept;
        size_t n_articulos() const noexcept;

        //Metodos para relacionar/borrar relacion con objetos Tarjeta
        void es_titular_de(Tarjeta& t);
        void no_es_titular_de(Tarjeta& t);
        //Metodo para relacionar/borrar relacion con objetos Articulo
        void compra(Articulo& art, size_t cant = 1);

        //Clase de excepcion
        class Id_duplicado{
            public:
                Id_duplicado(const Cadena& i);
                const Cadena& idd() const noexcept;
            private:
                Cadena idd_;
        };
        //Para imprimir la clave, el operador de insercion de flujo debe
ser friend
        friend std::ostream& operator <<(std::ostream& os, const Usuario&
 u) noexcept;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    private:

        const Cadena id_, nombre_, apellidos_, direccion_;
        Clave clave_;
        Tarjetas tarjetas_;
        Articulos articulos_;

        //Set que guarda todos los usuarios
        static std::unordered_set<Cadena> usuarios_;

};
//Mostrar los articulos que tiene en el carro el usuario u
void mostrar_carro(std::ostream& os, const Usuario& u) noexcept;

//Metodos Inline
inline const Cadena& Usuario::id() const noexcept{return id_;}
inline const Cadena& Usuario::nombre() const noexcept{return nombre_;}
inline const Cadena& Usuario::apellidos() const noexcept{return apellidos
_;}
inline const Cadena& Usuario::direccion() const noexcept{return direccion
_;}
inline const Usuario::Tarjetas& Usuario::tarjetas() const noexcept{return
 tarjetas_;}
inline const Usuario::Articulos& Usuario::compra() const noexcept{return
articulos_;}
inline size_t Usuario::n_articulos() const noexcept{return articulos_.siz
e();}

//Metodos Inline de la clase de excepcion
inline Usuario::Id_duplicado::Id_duplicado(const Cadena& i):idd_(i){}
inline const Cadena& Usuario::Id_duplicado::idd() const noexcept{return i
dd_;}

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Usuario.cpp:

```
#include "usuario.hpp"


//Declaracion adelantada
std::unordered_set<Cadena> Usuario::usuarios_;

//Metodos de la Clase Clave-----------------------------------------------
-------------------------

//Constructor
Clave::Clave(const char* c){
    char* cad;
    //Cadena corta --> ERROR
    if(strlen(c) < 5) {
        Clave::Incorrecta corta(Clave::CORTA);
        throw corta;
    }
    else{
        //Caracteres disponibles para encriptar
        const char* cjto_caracteres = "./abcdefghijlmnopqrstuvwxyzABCDEFG
HIJKLMNOPQRSTUVWXYZ0123456789";
        //Creamos salt
        char salt[2];
        salt[0] = cjto_caracteres[rand() % strlen(cjto_caracteres)];
        salt[1] = cjto_caracteres[rand() % strlen(cjto_caracteres)];

        //Comprobamos que la encriptacion se haga correctamente, si no es
 asi --> ERROR
        if((cad = (crypt(c,salt))) == NULL){
            Clave::Incorrecta cryp(Clave::ERROR_CRYPT);
            throw cryp;
        }else //Asignamos al atributo la clave encriptada
            clave_ = cad;
    }


}

//Comparacion de una cadena de bajo nivel introducida con una ya cifrada
bool Clave::verifica(const char* c) const{
    return !strcmp(crypt(c, clave_.c_str()), clave_.c_str());
}

//Metodos de la clase Usuario---------------------------------------------
-------------------------

//Constructor
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
Usuario::Usuario(const Cadena& i, const Cadena& n, const Cadena& a, const
 Cadena& d, const Clave& c): id_{i}, nombre_{n}, apellidos_{a}, direccion
_{d}, clave_{c}{
    //Usuario ya en el sistema --> ERROR
    if(!usuarios_.insert(i).second){
        Usuario::Id_duplicado iddup(i);
        throw iddup;
    }
}

//Destructor
Usuario::~Usuario(){
    //Anulamos todas las tarjetas
    for(auto& kv:tarjetas_)
        kv.second->anula_titular();
    //Borramos al usuario del set de usuarios
    usuarios_.erase(id_);
}

//Vincular Usuario--Tarjeta
void Usuario::es_titular_de(Tarjeta& t){
    //Comprobamos que la tarjeta sea del usuario
    if(t.titular() == this)
        //La añadimos a las tarjetas del usuario
        tarjetas_[t.numero()] = &t;
}

//Desvincular Usuario--Tarjeta
void Usuario::no_es_titular_de(Tarjeta& t){
    //Borramos la tarjeta de lasque almacena el usuario (la tarjeta sigue
 <viva>)
    tarjetas_.erase(t.numero());
    t.anula_titular();
    t.activa(true);
}

//Comprar articulo (Vincular/Desvincular Usuario--Articulo)
void Usuario::compra(Articulo& art, size_t cant){
    //Buscamos el articulo en el carrito
    auto en_cesta = articulos_.find(&art);
    //No esta en el carrito --
> Añadimos el articulo si sus unidades son distintas de 0
    if(en_cesta == articulos_.end()){//Nuevo articulo
        if(cant > 0)
            articulos_[&art] = cant;

    }else{//Articulo ya en cesta
        if(cant > 0)//Sumamos cantidades
            en_cesta->second = cant;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        else //Borramos articulo
            articulos_.erase(en_cesta);
    }
}

//Operador de insercion en flujo
std::ostream& operator <<(std::ostream& os, const Usuario& u) noexcept{
    //datos del usuario
    os << u.id_ << " [" << u.clave_.clave() << "] "  << u.nombre_ << " "
<< u.apellidos_ << std::endl;
    os << u.direccion_ << std::endl;
    //tarjetas
    os << "Tarjetas: " << std::endl;
    for(auto& i:u.tarjetas_)
        os << *i.second << std::endl;
    return os;
}

//Mostrar los articulos que aparecen en el atributo articulos_
void mostrar_carro(std::ostream& os, const Usuario& u) noexcept{
    //Datos del usuario y numero de elementos en el carrito
    os << "Carrito de la compra de " << u.id() << " [Artículos: " << u.n_
articulos() << "]" << std::endl;
    os << "Cant.\tArt." <<std::endl;
    os << "==================================================================
=====================" << std::endl;
    //elementos y cantidades
    for(auto& i:u.compra())
        os << i.second << "\t" << *i.first << std::endl;

    os << std::endl;

}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Usuario-Pedido.hpp:

```
#ifndef _USUARIO_PEDIDO_HPP_
#define _USUARIO_PEDIDO_HPP_

#include <set>
#include <map>

class Usuario;
class Pedido;

class Usuario_Pedido{
    public:
        //Tipo Pedidos
        typedef std::set<Pedido*> Pedidos;
        //Asociacion
        void asocia(Usuario& u, Pedido& p);
        void asocia(Pedido& p, Usuario& u);
        //Observadores
        Pedidos pedidos(const Usuario& u)const noexcept;
        const Usuario* cliente(const Pedido& u) const noexcept;

    private:
        //Diccionarios
        typedef std::map<Usuario*, Pedidos> UsuPed;
        typedef std::map<Pedido*, Usuario*> PedUsu;

        UsuPed up;
        PedUsu pu;
};

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Usuario-Pedido.cpp:

```
#include "usuario-pedido.hpp"

//Asociacion-------------------------------------------------------------
-------------------------
void Usuario_Pedido::asocia(Usuario& u, Pedido& p){
    //Añadimos al dicionario los valores
    up[&u].insert((Pedido*)&p);
    pu[&p] = (Usuario*)&u;
}

//En la función sobrecargada se llama a la otra funcion asocia
void Usuario_Pedido::asocia(Pedido& p, Usuario& u){
    asocia(u,p);
}

//Observadores-----------------------------------------------------------
-------------------------
Usuario_Pedido::Pedidos Usuario_Pedido::pedidos(const Usuario& u)const no
except{
    //Buscamos el usuario en el diccionario
    auto i = up.find((Usuario*)&u);
    if(i != up.end()) //Esta en el diccionario
        //Devolvemos el set asociado al usuario
        return i->second;
    else //No estaba en el diccionario
        //Devolvemos un set vacio
        return Usuario_Pedido::Pedidos{};
}

const Usuario* Usuario_Pedido::cliente(const Pedido& p) const noexcept{
    //Buscamos en el diccionario
    auto i = pu.find((Pedido*)&p);
    if(i != pu.end())//Esta en el diccionario
        //Devolvemos el usuario asociado
        return i->second;
    else{ //No esta en el diccionario
        //Devolvemos un puntero vacio
        return nullptr;
    }
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Pedido-Articulo.hpp:

```
#ifndef _PEDIDO_ARTICULO_HPP_
#define _PEDIDO_ARTICULO_HPP_
#include <iostream>
#include <map>
#include <iomanip>


#include "pedido.hpp"
#include "articulo.hpp"

class Pedido;

//LINEAPEDIDO-------------------------------------------------------------
-------------------------
class LineaPedido{
    public:
        //Constructor
        explicit LineaPedido(double p, unsigned cantidad = 1);
        //Observadores
        const double precio_venta() const noexcept;
        const unsigned cantidad() const noexcept;

    private:
        double precio_venta_;
        unsigned cantidad_;
};
//Insercion en flujo
std::ostream& operator <<(std::ostream& os, const LineaPedido& lp);

//Funciones inline de lineapedido
inline LineaPedido::LineaPedido(double p, unsigned cant): precio_venta_{p
}, cantidad_{cant}{}
inline const double LineaPedido::precio_venta() const noexcept {return pr
ecio_venta_;}
inline const unsigned LineaPedido::cantidad() const noexcept {return cant
idad_;}

//ORDENACION--------------------------------------------------------------
-------------------------

class OrdenaArticulos: public std::binary_function<Articulo*, Articulo*,
bool>{
    public:
        bool operator()(const Articulo* a1, const Articulo* a2)const noex
cept;
};

class OrdenaPedidos: public std::binary_function<Pedido*, Pedido*, bool>{
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    public:
        bool operator()(const Pedido* p1, const Pedido* p2)const noexcept
;
};


//PEDIDOARTICULO-------------------------------------------------------
-------------------------
class Pedido_Articulo{
    public:
        //Tipos definidos
        typedef std::map<Articulo*, LineaPedido, OrdenaArticulos> ItemsPe
dido;
        typedef std::map<Pedido*, LineaPedido, OrdenaPedidos> Pedidos;
        //Metodos de asociacion
        void pedir(Pedido& ped, Articulo& art, double pr, unsigned cant =
 1);
        void pedir(Articulo& art, Pedido& ped, double pr, unsigned cant =
 1);
        //Observadores
        ItemsPedido detalle(const Pedido& ped) const;
        Pedidos ventas(const Articulo& art) const;
        //Metodos mostrar
        void mostrarDetallePedidos(std::ostream& os) const noexcept;
        void mostrarVentasArticulos(std::ostream& os) const noexcept;
    private:
        //Diccionarios
        typedef std::map<Pedido*, ItemsPedido, OrdenaPedidos> PedArt;
        typedef std::map<Articulo*, Pedidos, OrdenaArticulos> ArtPed;
        PedArt pa;
        ArtPed ap;
};
//Insercion en flujo
std::ostream& operator <<(std::ostream& os, const Pedido_Articulo::ItemsP
edido& ip);
std::ostream& operator <<(std::ostream& os, const Pedido_Articulo::Pedido
s& p);

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Pedido-Articulo.cpp:

```
#include "pedido-articulo.hpp"

//LINEAPEDIDO-----------------------------------------------------------
-------------------------
//Insercion en flujo LINEAPEDIDO
std::ostream& operator <<(std::ostream& os, const LineaPedido& lp){
    std::locale::global(std::locale(""));
    os << std::fixed << std::setprecision(2) << lp.precio_venta() << " €\
t" << lp.cantidad();
    return os;
}

//PEDIDO----------------------------------------------------------------
-------------------------
//Metodos de asociacion
void Pedido_Articulo::pedir(Pedido& ped, Articulo& art, double pr, unsign
ed cant){
    pa[&ped].insert(std::make_pair(&art, LineaPedido(pr,cant)));
    ap[&art].insert(std::make_pair(&ped,LineaPedido(pr,cant)));

}
//Sobrecarga: Llama al otro metodo pedir
void Pedido_Articulo::pedir(Articulo& art, Pedido& ped, double pr, unsign
ed cant){
    pedir(ped,art,pr,cant);
}
//Observadores
Pedido_Articulo::ItemsPedido Pedido_Articulo::detalle(const Pedido& ped)
const{
    //Busqueda en el diccionario
    auto i = pa.find((Pedido*)&ped);
    if(i != pa.end())//Existe en el diccionario
        //Devuelve ItemsPedido asociado
        return i->second;
    else
        //Devuelve temsPedido vacio
        return ItemsPedido{};
}

Pedido_Articulo::Pedidos Pedido_Articulo::ventas(const Articulo& art) con
st{
    //Busqueda en el diccionario
    auto i = ap.find((Articulo*)&art);
    if(i != ap.end())//Existe en el diccionario
        //Devuelve los pedidos asociados
        return i->second;
    else
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        //Devuelve Pedidos vacio
        return Pedidos{};
}
//Insercion en flujo ItemsPedido
std::ostream& operator <<(std::ostream& os, const Pedido_Articulo::ItemsP
edido& ip){
    //Para calcular el total
    double total = 0.;
    os << " PVP\tCantidad\tArtículo" << std::endl;
    os << std::setfill('=') << std::setw(75) << " " << std::endl;
    //Imprimimos todo el contenido
    for(auto& i:ip){
        total += (i.second.precio_venta() * i.second.cantidad());
        os << i.second << "\t\t";
        os << "[" << i.first->referencia() << "] " << i.first-
>titulo() << std::endl;;

    }
    os << std::setfill('=') << std::setw(75) << " " << std::endl;
    os << "Total\t" << total << " €" << std::endl << std::endl << std::en
dl;
    return os;
}
//Insercion en flujo Pedidos
std::ostream& operator <<(std::ostream& os, const Pedido_Articulo::Pedido
s& p){
    //Para calcular el total de precio y de cantidad de articulos
    double total = 0.;
    unsigned cantidad = 0;
    os << "[Pedidos: " << p.size() << "]" << std::endl;
    os << std::setfill('=') << std::setw(75) << " " << std::endl;
    os << " PVP\tCantidad\tFecha" << std::endl;
    os << std::setfill('=') << std::setw(75) << " " << std::endl;
    //Imprimimos todo el contenido
    for(auto& i:p){
        total += i.second.precio_venta() * i.second.cantidad();
        cantidad += i.second.cantidad();
        os << i.second << "\t\t" << i.first->fecha() << std::endl;
    }
    os << std::setfill('=') << std::setw(75) << " " << std::endl;
    os << total << " €\t" << cantidad << std::endl << std::endl << std::e
ndl;
    return os;
}


//Funciones mostrar
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
void Pedido_Articulo::mostrarDetallePedidos(std::ostream& os) const noexc
ept{
    //Para calcular el total ventas
    double precio = 0.;
    for(auto& i:pa){
        os << "Pedido núm. " << i.first->numero() << std::endl;
        os << "Cliente: " << i.first->tarjeta()->titular()-
>id() << "\tFecha: " << i.first->fecha() << std::endl;
        //Imprimimos ItemsPedido
        os << detalle(*i.first);
        precio += i.first->total();
    }
    os << std::endl << "TOTAL VENTAS\t" << precio << " €" << std::endl;
}

void Pedido_Articulo::mostrarVentasArticulos(std::ostream& os) const noex
cept{
    for(auto& i:ap){
        os << "Ventas de " << "[" << i.first-
>referencia() << "] " << i.first->titulo() << std::endl;
        //Imprimimos Pedidos
        os << ventas(*i.first);
    }
}
//-----------------------------------------------------------------------
-------------------------
bool OrdenaArticulos::operator()(const Articulo* a1, const Articulo* a2)c
onst noexcept{return a1->referencia() < a2->referencia();}
bool OrdenaPedidos::operator()(const Pedido* p1, const Pedido* p2)const n
oexcept{return p1->numero() < p2->numero();}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

Práctica 4:

·Fecha.hpp:

```
#ifndef _FECHA_HPP_
#define _FECHA_HPP_

#include <iostream>

class Fecha{
    public:
        //Constructores
        explicit Fecha(int d = 0, int m = 0, int a = 0); //Con 3, 2, 1 y
ningún parámetro
        Fecha(const Fecha& f) = default; //Ctor. de copia
        Fecha(const char* c); //A partir de una cadena de caracteres de f
ormato dd/mm/aaaa
        Fecha& operator = (const Fecha&) = default; //Operador asignación

        //Conversion a const *char
        const char* cadena() const;

        //Constantes
        static const int AnnoMinimo = 1902;
        static const int AnnoMaximo = 2037;

        //Clase Fecha Invalida
        class Invalida{
            public:
                Invalida(const char* inf);
                const char* por_que() const noexcept;
            private:
                const char* info_;
        };

        //Observadores
        int dia() const noexcept;
        int mes() const noexcept;
        int anno() const noexcept;
        //Operadores
        Fecha& operator ++();
        Fecha operator ++(int);
        Fecha& operator --();
        Fecha operator --(int);
        Fecha& operator +=(int i);
        Fecha& operator -=(int i);

        //Operadores Aritméticos
        Fecha operator +(int i) const;
        Fecha operator -(int i) const;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        //Destructor
        ~Fecha() = default;

    private:
        int dia_, mes_, anno_;
        void comprueba();
        int ultimo_dia() const;

};



//Operadores Lógicos
bool operator ==(const Fecha& f1, const Fecha& f2) noexcept;
bool operator !=(const Fecha& f1, const Fecha& f2) noexcept;
bool operator <(const Fecha& f1, const Fecha& f2) noexcept;
bool operator >(const Fecha& f1, const Fecha& f2) noexcept;
bool operator <=(const Fecha& f1, const Fecha& f2) noexcept;
bool operator >=(const Fecha& f1, const Fecha& f2) noexcept;

//Entrada/Salida-------------------------------------------------------
--------------------------
std::ostream& operator <<(std::ostream& os, const Fecha& f) noexcept;
std::istream& operator >>(std::istream& is, Fecha& f);

//Métodos Inline-------------------------------------------------------
--------------------------

inline int Fecha::dia() const noexcept{return dia_;}
inline int Fecha::mes() const noexcept{return mes_;}
inline int Fecha::anno() const noexcept{return anno_;}

//Métodos Inline INVALIDA----------------------------------------------
--------------------------
inline Fecha::Invalida::Invalida(const char* inf): info_(inf){}
inline const char* Fecha::Invalida::por_que() const noexcept{return info_
;}


#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Fecha.cpp:

```
#include "fecha.hpp"
#include <ctime>
#include <cstdio>
#include <iostream>
#include <locale>

//FUNCIONES PRIVADAS ----------------------------------------------------
-----------------------------
//Funcion que lanza excepciones si el dia o el año es incorrecto (ultimo_
dia() es llamada desde aqui)
void Fecha::comprueba(){
    //menor que el ultimo dia del mes de la fecha parametro implicito y m
ínimo 1
    if(dia_ > ultimo_dia() || dia_ < 1){
        Fecha::Invalida d_inval("Dia Incorrecto");
        throw d_inval;
    }
    //anno entre los annos validos
    if(anno_ < Fecha::AnnoMinimo || anno_ > Fecha::AnnoMaximo){
        Fecha::Invalida a_inval("Año Incorrecto");
        throw a_inval;
    }

}
//Función privada que, a partir de un mes, devuelve su ultimo dia o lanza
 una excepcion si es incorrecto
int Fecha::ultimo_dia() const {
    //Enero, marzo, mayo, julio, agosto, octubre y diciembre --> 31 dias
    if(mes_ == 1 || mes_ == 3 || mes_ == 5 || mes_ == 7 || mes_ == 8 || m
es_ == 10 || mes_ == 12)
        return 31;
    //Abril, junio, septiembre y noviembre --> 30 dias
    else if(mes_ == 4 || mes_ == 6 || mes_ == 9 || mes_ == 11)
        return 30;
    //Febrero --> 28 o 29 segun sea bisiesto o no
    else if(mes_ == 2){ //¿Es bisiesto?
        if(anno_ % 4 == 0 && (anno_ % 400 == 0 || anno_ % 100 != 0))
            return 29;
        else
            return 28;
    //El mes no esta en [1-12]
    }else{
        Fecha::Invalida inval("Mes Incorrecto");
        throw inval;
        return 0;
    }
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
//CONSTRUCTORES-------------------------------------------------------------
------------------------------
Fecha::Fecha(int d, int m, int a): dia_{d}, mes_{m}, anno_{a}{
    //Obtenemos fecha actual por si existen 0 en dia, mes o anno
    std::time_t tiempo_calendario = std::time(nullptr);
    std::tm* tiempo_descompuesto = std::localtime(&tiempo_calendario);
    //Cambio de 0 a día, mes o año actual
    if(anno_ == 0)
        anno_ = tiempo_descompuesto->tm_year +1900;

    if(mes_ == 0)
        mes_ = tiempo_descompuesto->tm_mon + 1;

    if (dia_ == 0)
        dia_ = tiempo_descompuesto->tm_mday;
    //Se comprueba si la fecha es valida
    comprueba();
}

Fecha::Fecha (const char* c){
    //Tomamos los valores directamente de la cadena
    if(sscanf(c,"%d/%d/%d",&dia_,&mes_,&anno_) != 3){
        //Si es distinto de 3, no hemos leído día, mes y año (Falta uno d
e ellos o hay otros caracteres distintos de '/')
        Fecha::Invalida format("Formato de la cadena incorrecto.");
        throw format;
    }
    //Tomamaos fecha actual por si existen 0 en dia, mes o anno
    std::time_t tiempo_calendario = std::time(nullptr);
    std::tm* tiempo_descompuesto = std::localtime(&tiempo_calendario);
    //Cambio de 0 a día, mes o año actual
    if(anno_ == 0)
        anno_ = tiempo_descompuesto->tm_year +1900;

    if(mes_ == 0)
        mes_ = tiempo_descompuesto->tm_mon + 1;

    if (dia_ == 0)
        dia_ = tiempo_descompuesto->tm_mday;
    //Se comprueba si la fecha es valida
    comprueba();
}


//CONVERSOR FECHA ---> CONST CHAR* ------------------------------------
-------------------------
const char* Fecha::cadena() const{
    //Idioma espannol
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    std::locale::global(std::locale("es_ES.utf8"));
    char *aux = new char[100];
    //Creamos una estructura tm e introducimos dia, mes y anno en ella
    std::tm f{};

    f.tm_mday = dia_;
    f.tm_mon = mes_ - 1;
    f.tm_year = anno_ - 1900;
    //Formatea la fecha y la devuelve
    mktime(&f);
    strftime(aux,100,"%A %e de %B de %Y",&f);
    return aux;

}

//OPERADORES-------------------------------------------------------------
-------------------------
Fecha& Fecha::operator +=(int i){//f += i
    //Creamos una estructura con la fecha ya operada con el entero y la f
ormateamos con mktime(p. ej. cambiamos de 32-4-2020 a 2-5-2020)
    std::tm f{};
    f.tm_mday = dia_ + i;
    f.tm_mon = mes_ - 1;
    f.tm_year = anno_ - 1900;
    std::mktime(&f);

    //Pasamos la fecha formateada a los atributos del parametro implicito
    dia_ = f.tm_mday;
    mes_ = f.tm_mon + 1;
    anno_ = f.tm_year + 1900;
    comprueba();
    return *this;
}

Fecha& Fecha::operator -=(int i){ //f -= i
    *this += -i;
    return *this;
}

Fecha& Fecha::operator ++(){ //++f

    *this += 1;
    return *this;
}

Fecha Fecha::operator ++(int){ //f++
    Fecha *f = new Fecha(*this);
    *this += 1;
    return *f;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
}

Fecha& Fecha::operator --(){ //--f
    *this += -1;
    return *this;
}

Fecha Fecha::operator --(int){ //f--
    Fecha *f = new Fecha(*this);
    *this += -1;
    return *f;
}

//OPERADORES ARTIMÉTICOS-----------------------------------------------
-------------------------
Fecha Fecha::operator +(int i) const{//f + i
    Fecha f(*this);
    return f += i;
}

Fecha Fecha::operator -(int i) const{//f - i
    Fecha f(*this);
    return f += -i;
}

//OPERADORES LÓGICOS---------------------------------------------------
-------------------------
bool operator ==(const Fecha& f1, const Fecha& f2) noexcept{//f1 == f2
    //Comparamos campo a campo
    return (f1.dia() == f2.dia() && f1.mes() == f2.mes() && f1.anno() ==
f2.anno());
}
bool operator !=(const Fecha& f1, const Fecha& f2) noexcept{//f1 != f2
    return !(f1 == f2);
}
bool operator <(const Fecha& f1, const Fecha& f2) noexcept{// f1 < f2
    /* Para devolver true, debe ser una de estas opciones:
    *   -Anno del primero menor que anno del segundo
    *   -Annos iguales, mes del primero menor que mes del segundo
    *   -Annos y meses iguales, dia del primero menor que dia del segundo
    */
    return (f1.anno() < f2.anno() || (f1.anno() == f2.anno() && (f1.mes()
 < f2.mes() || (f1.mes() == f2.mes() && f1.dia() < f2.dia()))));
}
bool operator >(const Fecha& f1, const Fecha& f2) noexcept{//f1 > f2
    return f2 < f1;
}
bool operator <=(const Fecha& f1, const Fecha& f2) noexcept{//f1 <= f2
    return !(f2 < f1);
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
}
bool operator >=(const Fecha& f1, const Fecha& f2) noexcept{// f1 >= f2
    return !(f1 < f2);
}

//ENTRADA/SALIDA-------------------------------------------------------
-------------------------
std::ostream& operator <<(std::ostream& os, const Fecha& f) noexcept{ //s
alida
    //Imprimimos la fecha formateada ("nombre_dia_semana, dia_num de nomb
re_mes de anno_num")
    os << f.cadena();
    return os;
}

std::istream& operator >>(std::istream& is, Fecha& f){ //entrada
    char linea[11];
    is.getline(linea, 11);
    try{
        //Llamada a ctor. de fecha a partir de una cadena, y al operador
de asig.
        f = linea;
    }catch(Fecha::Invalida e){
        //Failbit
        is.setstate(std::ios::failbit);
        throw e;
    }
    return is;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Cadena.hpp:

```
#ifndef _CADENA_HPP_
#define _CADENA_HPP_

#include <iostream>
#include <string>

class Cadena{
    public:
        //Constructores
        explicit Cadena(size_t tam = 0, char c = ' ');
        Cadena(const Cadena& cad);
        Cadena(const char* cad);
        Cadena (Cadena&& cad);

        //Operadores de asignación
        Cadena& operator =(const Cadena& cad);
        Cadena& operator =(Cadena&& cad);

        //Conversor a const char*
        const char* c_str() const noexcept;

        //Métodos Observadores
        size_t length() const noexcept;

        //Operador de Concatenación de Cadenas
        Cadena& operator +=(const Cadena& cad);


        //Operadores de Índice
        char& operator[](size_t i);
        const char& operator[](size_t i) const;
        char& at(size_t i);
        const char& at(size_t i) const;

        //Función Subcadena
        Cadena substr(size_t i, size_t tam) const;

        //Iteradores
        typedef char* iterator;
        typedef const char* const_iterator;
        typedef std::reverse_iterator<iterator> reverse_iterator;
        typedef std::reverse_iterator<const_iterator> const_reverse_itera
tor;

        //Funciones de iteradores
        iterator begin() noexcept;
        const_iterator begin() const noexcept;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        reverse_iterator rbegin() noexcept;
        const_reverse_iterator rbegin() const noexcept;
        const_iterator cbegin() const noexcept;
        const_reverse_iterator crbegin() const noexcept;
        iterator end() noexcept;
        const_iterator end() const noexcept;
        reverse_iterator rend() noexcept;
        const_reverse_iterator rend() const noexcept;
        const_iterator cend() const noexcept;
        const_reverse_iterator crend() const noexcept;

        //Destructor
        ~Cadena();

    private:
        char* s_;
        size_t tam_;

};

//Operadores Lógicos
bool operator ==(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator !=(const Cadena& cad1, const Cadena& cad2)noexcept;
bool operator <(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator >(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator <=(const Cadena& cad1, const Cadena& cad2) noexcept;
bool operator >=(const Cadena& cad1, const Cadena& cad2) noexcept;

//Operador de concatenación de cadenas
Cadena operator +(const Cadena& cad1, const Cadena& cad2);

//Entrada/Salida
std::ostream& operator <<(std::ostream& os, const Cadena& cad) noexcept;
std::istream& operator >>(std::istream& is, Cadena& cad);
//Funciones Inline-------------------------------------------------------
-------------------------
inline size_t Cadena::length() const noexcept{
    return tam_;
}

inline char& Cadena::operator[](size_t i){
    return s_[i];
}
inline const char& Cadena::operator[](size_t i) const {
    return s_[i];
}

inline const char* Cadena::c_str() const noexcept{
    return s_;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
}


inline Cadena::iterator Cadena::begin() noexcept{
    return s_;
}
inline Cadena::const_iterator Cadena::begin() const noexcept{
    return const_iterator(s_);
}
inline Cadena::iterator Cadena::end() noexcept{
    return s_+tam_;
}
inline Cadena::const_iterator Cadena::end() const noexcept{
    return const_iterator(s_+tam_);
}
inline Cadena::reverse_iterator Cadena::rbegin() noexcept{
    return reverse_iterator(end());
}
inline Cadena::const_reverse_iterator Cadena::rbegin() const noexcept{
    return const_reverse_iterator(end());
}
inline Cadena::const_iterator Cadena::cbegin() const noexcept{
    return const_iterator(s_);
}
inline Cadena::const_reverse_iterator Cadena::crbegin() const noexcept{
    return const_reverse_iterator(end());
}
inline Cadena::reverse_iterator Cadena::rend() noexcept{
    return reverse_iterator(begin());
}
inline Cadena::const_reverse_iterator Cadena::rend() const noexcept{
    return const_reverse_iterator(begin());
}
inline Cadena::const_iterator Cadena::cend() const noexcept{
    return const_iterator(s_+tam_);
}
inline Cadena::const_reverse_iterator Cadena::crend() const noexcept{
    return const_reverse_iterator(begin());
}
//Para P2 y ss
//Especializacion de la plantilla hash<T> para definir la
//funcion hash a utilizar con contenedores desordenados de
//Cadena. unordered_[st|map|multiset|multimap]
namespace std{ //Estaremos dentro del espacio de nombres std
    template <>//Es una especializacion de una plantilla de Cadena
    struct hash<Cadena>{ //Es una clase con solo un operador publico
        size_t operator() (const Cadena& cad) const //El operador funcion
        {
            hash<string> hs;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
            const char* p = cad.c_str();
            string s(p);
            size_t res = hs(s);
            return res;
        }
    };


}


#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Cadena.cpp:

```
#include "cadena.hpp"
#include <cstring>
#include <iostream>
#include <stdexcept>

//Constructores-----------------------------------------------------------
-------------------------
Cadena::Cadena(size_t tam, char c): s_{new char[tam+1]}, tam_{tam}{ //Pre
det.
    for(size_t i = 0; i < tam_; i++)
        s_[i] = c;
    s_[tam_] = '\0';
}

Cadena::Cadena(const Cadena& cad): s_{new char[cad.tam_ + 1]}, tam_{cad.t
am_}{ //Copia
    strcpy(s_,cad.s_);
}

Cadena::Cadena(const char* cad): s_{new char[strlen(cad)+1]}, tam_{strlen
(cad)}{ //A partir de cadena bajo nivel
    strcpy(s_, cad);
}

Cadena::Cadena(Cadena&& cad): s_{cad.s_}, tam_{cad.tam_}{ //Movimiento
    cad.tam_ = 0;
    cad.s_ = nullptr;
}

//Operador de Asgnación---------------------------------------------------
-------------------------
Cadena& Cadena::operator =(const Cadena& cad){ //Op. Asig. por copia
    if(this != &cad){ //Evitamos autoasignación
        tam_ = cad.tam_;
        delete[] s_;
        s_ = new char[tam_ + 1];
        strcpy(s_, cad.s_);
    }
    return *this;
}

Cadena& Cadena::operator = (Cadena&& cad){ //Op. Asig. Semántica de Movim
iento
    if(this != &cad){
        tam_ = cad.tam_;
        delete[] s_;
        s_ = cad.s_;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        cad.s_ = nullptr;
        cad.tam_ = 0;
    }
    return *this;

}
//Operadores de Concatenación-----------------------------------------
-------------------------
Cadena& Cadena::operator +=(const Cadena& cad){
    char* aux = new char[tam_+1];
    strcpy(aux,s_);

    tam_ += cad.tam_;

    delete[] s_;
    s_ = new char[tam_ + 1];

    strcpy(s_, aux);
    strcat(s_, cad.s_);

    delete[] aux;
    return *this;
}

Cadena operator +(const Cadena& cad1, const Cadena& cad2){
    Cadena aux(cad1);
    return aux += cad2;

}

//Operadores Lógicos--------------------------------------------------
-------------------------
bool operator ==(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !strcmp(cad1.c_str(), cad2.c_str());
}

bool operator !=(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !(cad1 == cad2);
}

bool operator <(const Cadena& cad1, const Cadena& cad2) noexcept{
    //Si alfabéticamente es menor, strcmp devuelve un valor negativo
    return (strcmp(cad1.c_str(), cad2.c_str()) < 0);
}

bool operator >(const Cadena& cad1, const Cadena& cad2) noexcept{
    return cad2 < cad1;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
bool operator <=(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !(cad2 < cad1);
}

bool operator >=(const Cadena& cad1, const Cadena& cad2) noexcept{
    return !(cad1 < cad2);
}
//Funciones AT-------------------------------------------------------------
-------------------------
char& Cadena::at(size_t i){
    if(i < tam_)
        return s_[i];
    else
        throw std::out_of_range("Funcion at(): fuera de limites.");
}

const char& Cadena::at(size_t i) const{
    if(i < tam_)
        return s_[i];
    else
        throw std::out_of_range("Funcion at(): fuera de limites.");

}

Cadena Cadena::substr(size_t i, size_t tam) const{
    //Al ser enteros sin signo, si introducimos un número neg. da uno pos
itivo muy grande (Deborda).
    if(i+tam > tam_ || tam > tam_ || i > tam_)
        throw std::out_of_range("Funcion substr(): fuera de limites.");
    else{
        Cadena aux(tam);
        strncpy(aux.s_, s_ + i, tam);
        aux.s_[tam] = '\0';
        return aux;
    }
}
//Entrada/Salida-----------------------------------------------------------
-------------------------
std::ostream& operator <<(std::ostream& os, const Cadena& cad) noexcept{
//Salida
    //Imprimimos la cadena de bajo nivel (atributo privado s_)
    os << cad.c_str();
    return os;
}
std::istream& operator >>(std::istream& is, Cadena& cad){ //Entrada
    //Creamos una cadena de bajo nivel de 33 caracteres (32 + '\0')
    char linea[33] = "";
    is.width(33); //32 caracteres + carácter nulo
    is >> linea;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    //Llamada al ctor de cadena a partir de una de bajo nivel y al operad
or de asig.
    cad = linea;
    return is;

}
//Destructor-------------------------------------------------------------
--------------------------
Cadena::~Cadena(){
    //Borramos el contenido de la cadena de bajo nivel, el tamanno se bor
raria automaticamente
    delete[] s_;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Makefile (P1):

```
CXX = clang++
CXXFLAGS = -g -Wall -std=c++11 -pedantic
CPPFLAGS = -DP1 -I../Tests-auto -I.
VPATH = ../Tests-auto:.

#Archivos ejecutables
EJEC = test-P0-auto test-P0-consola

all: test-P0-auto test-P0-consola

fecha.o: fecha.cpp fecha.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) fecha.cpp

cadena.o: cadena.cpp cadena.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) cadena.cpp

test-fechacadena-consola.o: test-fechacadena-
consola.cpp cadena.hpp fecha.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) test-fechacadena-consola.cpp

test-P0-consola: test-fechacadena-consola.o fecha.o cadena.o
    $(CXX) -o $@ $^

test-P0-auto: test-caso0-fecha-auto.o test-caso0-cadena-auto.o test-
auto.o cadena.o fecha.o
    $(CXX) -o  $@ $^

test-caso0-fecha-auto.o test-caso0-cadena-auto.o test-auto.o: test-caso0-
fecha-auto.cpp test-caso0-cadena-auto.cpp test-auto.cpp test-
auto.hpp fecha.hpp cadena.hpp
    $(CXX) -c $(CXXFLAGS) $(CPPFLAGS) $^

fecha_cadena_check:
    $(MAKE) -f Make_check.mk $@
clean:
    $(RM) $(EJEC) *.o *.gch core  *~
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Articulo.hpp:

```
#ifndef _ARTICULO_HPP_
#define _ARTICULO_HPP_

#include "cadena.hpp"
#include "fecha.hpp"
#include <set>      //Para el tipop Autores
#include <iostream> //ostream
#include <locale>  //Para imprimir el precio con coma
#include <iomanip> //setfill, setw

//Clase Autor-------------------------------------------------------------
-------------------------
class Autor{
    public:
        //Constructor
        Autor(const Cadena& nom, const Cadena& ape, const Cadena& dir);
        //observadores
        const Cadena& nombre() const noexcept;
        const Cadena& apellidos() const noexcept;
        const Cadena& direccion() const noexcept;
    private:
        Cadena nombre_;
        Cadena apellidos_;
        Cadena direccion_;
};
//Metodos Inline de Autor
inline Autor::Autor(const Cadena& nom, const Cadena& ape, const Cadena& d
ir): nombre_{nom}, apellidos_{ape}, direccion_{dir}{}
inline const Cadena& Autor::nombre() const noexcept{return nombre_;}
inline const Cadena& Autor::apellidos() const noexcept{return apellidos_;
}
inline const Cadena& Autor::direccion() const noexcept{return direccion_;
}


//Clase Articulo----------------------------------------------------------
-------------------------
class Articulo{
    public:
        //Tipo autores
        typedef std::set<Autor*> Autores;
        //Constructor
        Articulo(const Autores& aut, const Cadena& r, const Cadena& t, co
nst Fecha& f, double p);

        //Observadores
        const Cadena& referencia() const noexcept;
        const Cadena& titulo() const noexcept;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        const Fecha& f_publi() const noexcept;
        const Autores& autores() const noexcept;
        //Observadores y sobrecargas no const
        double precio() const noexcept;
        double& precio() noexcept;

        //Clase de Excepcion
        class Autores_vacios{};

        //Metodos virtuales
        virtual void impresion_especifica(std::ostream& os) const = 0;
        virtual ~Articulo(){}

    private:
        const Autores autores_;
        const Cadena referencia_, titulo_;
        const Fecha f_publi_;
        double precio_;


};
//Operador de insercion en flujo de articulo
std::ostream& operator <<(std::ostream& os, const Articulo& art) noexcept
;


//Metodos Inline Articulo
inline const Articulo::Autores& Articulo::autores() const noexcept{return
 autores_;}
inline const Cadena& Articulo::referencia() const noexcept{return referen
cia_;}
inline const Cadena& Articulo::titulo() const noexcept{return titulo_;}
inline const Fecha& Articulo::f_publi() const noexcept{return f_publi_;}
inline double Articulo::precio() const noexcept{return precio_;}
inline double& Articulo::precio() noexcept{return precio_;}


//Clase LibroDigital-------------------------------------------------
-------------------------
class LibroDigital: public Articulo{
    public:
        //Constructor
        LibroDigital(const Autores& aut, const Cadena& r, const Cadena& t
, const Fecha& f, double p, const Fecha& f_exp);
        //Observador
        const Fecha& f_expir() const noexcept;
        //Impresion especifica libro digital
        void impresion_especifica(std::ostream& os) const;
    private:
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        const Fecha f_expir_;
};
//Metodos Inline de LibroDigital
inline LibroDigital::LibroDigital(const Autores& aut, const Cadena& r, co
nst Cadena& t, const Fecha& f, double p, const Fecha& f_exp): Articulo(au
t,r,t,f,p), f_expir_{f_exp}{}
inline const Fecha& LibroDigital::f_expir() const noexcept{return f_expir
_;}


//Clae ArticuloAlmacenable-----------------------------------------------
-------------------------
class ArticuloAlmacenable: public Articulo{
    public:
        //Constructor
        ArticuloAlmacenable(const Autores& aut, const Cadena& r, const Ca
dena& t, const Fecha& f, double p, size_t e = 0);
        //Observador
        size_t stock() const noexcept;
        //Modificador
        size_t& stock() noexcept;

    protected:
        size_t stock_;
};
//Metodos Inline de ArticuloAlmacenable
inline ArticuloAlmacenable::ArticuloAlmacenable(const Autores& aut, const
 Cadena& r, const Cadena& t, const Fecha& f, double p, size_t e): Articul
o(aut,r,t,f,p), stock_{e}{}
inline size_t ArticuloAlmacenable::stock() const noexcept{return stock_;}
inline size_t& ArticuloAlmacenable::stock() noexcept{return stock_;}


//Clase Libro------------------------------------------------------------
-------------------------
class Libro: public ArticuloAlmacenable{
    public:
        //Constructor
        Libro(const Autores& aut, const Cadena& r, const Cadena& t, const
 Fecha& f, double p, size_t paginas, size_t e = 0);
        //Observador
        size_t n_pag() const noexcept;
        //Impresion Especifica de Libro
        void impresion_especifica(std::ostream& os) const;
    private:
        const size_t n_pag_;
};
//Metodos Inline de Libro
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
inline Libro::Libro(const Autores& aut, const Cadena& r, const Cadena& t,
 const Fecha& f, double p, size_t paginas, size_t e):ArticuloAlmacenable(
aut,r,t,f,p,e), n_pag_{paginas}{}
inline size_t Libro::n_pag() const noexcept{return n_pag_;}


//Clase Cederron-------------------------------------------------------
-------------------------
class Cederron: public ArticuloAlmacenable{
    public:
        //Constructor
        Cederron(const Autores& aut, const Cadena& r, const Cadena& t, co
nst Fecha& f, double p, size_t tam, size_t e = 0);
        //Observador
        size_t tam() const noexcept;
        //Impresion Especifica de Cederron
        void impresion_especifica(std::ostream& os) const;
    private:
        const size_t tam_;
};
//Metodos Inline de Cederron
inline Cederron::Cederron(const Autores& aut, const Cadena& r, const Cade
na& t, const Fecha& f, double p, size_t tam, size_t e):ArticuloAlmacenabl
e(aut,r,t,f,p,e), tam_{tam}{}
inline size_t Cederron::tam() const noexcept{return tam_;}


#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Articulo.cpp:

```
#include "articulo.hpp"


//Metodos Clase Articulo

//Constructor
Articulo::Articulo(const Autores& aut, const Cadena& r, const Cadena& t,
const Fecha& f, double p) : autores_{aut}, referencia_{r}, titulo_{t}, f_
publi_{f}, precio_{p}{
    if(aut.empty())
        throw Articulo::Autores_vacios{};
}

//Insercion en flujo
std::ostream& operator <<(std::ostream& os, const Articulo& art) noexcept
{
    //Para poder usar la coma en el precio
    std::locale::global(std::locale(""));
    os << "[" << art.referencia() << "] " << "\"" << art.titulo() << "\",
 de ";
    auto i = art.autores().begin();
    os << (*i)->apellidos();
    while(++i != art.autores().end())
        os << ", " << (*i)->apellidos();
    os << ". " << art.f_publi().anno() << ". " << std::fixed << std::setp
recision(2) << art.precio() << " €" << std::endl << "\t";
    art.impresion_especifica(os);
    return os;
}

//Impresion Especifica
void LibroDigital::impresion_especifica(std::ostream& os) const{
    os << "A la venta hasta el " << f_expir_ << ".";
}

void Libro::impresion_especifica(std::ostream& os) const{
    os << n_pag_ << " págs., " << stock_ << " unidades.";
}

void Cederron::impresion_especifica(std::ostream& os) const{
    os << tam_ << " MB, " << stock_ << " unidades.";
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Tarjeta.hpp:

```
#ifndef _TARJETA_HPP_
#define _TARJETA_HPP_

#include "cadena.hpp"
#include "fecha.hpp"
#include "usuario.hpp"

#include <set>       //set numeros de tarjetas
#include <iostream>  //ostream
#include <cctype>    //isdigit, isspace
#include <iomanip>   //setfill, setw
#include <algorithm> //remove_if, find_if

//Declaración anticipada
class Usuario;

//Clase Numero-------------------------------------------------------------
-------------------------

class Numero{
    public:
        //Constructor
        Numero(const Cadena& cad);

        //Conversor a cadena de bajo nivel
        operator const char*()const noexcept;

        //Enumerado con las razones de numero incorrecto
        enum Razon{LONGITUD, DIGITOS, NO_VALIDO};

        //Clase Numero::Incorrecto
        class Incorrecto{
            public:
                Incorrecto(Numero::Razon r);
                Numero::Razon razon() const noexcept;
            private:
                Numero::Razon razon_;
        };

    private:
        Cadena numero_;
};
//Metodos Inline

//Constructor y observador clase incorrecto
inline Numero::Incorrecto::Incorrecto(Numero::Razon r):razon_{r}{}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
inline Numero::Razon Numero::Incorrecto::razon() const noexcept{return ra
zon_;}
//Conversor a const char*
inline Numero::operator const char*() const noexcept{return numero_.c_str
();}
//Operador de comparación
inline bool operator <(const Numero& n1, const Numero& n2){ return static
_cast<Cadena>(n1) < static_cast<Cadena>(n2);}

//Clase Tarjeta-------------------------------------------------------
-------------------------
class Tarjeta{
    public:

        //Constructores (suprimimos los de copia)
        Tarjeta(const Numero& n, Usuario& u, const Fecha& f);
        Tarjeta(const Tarjeta& t) = delete;
        Tarjeta& operator=(const Tarjeta& t) = delete;
        //Destructor
        ~Tarjeta();

        //Enumerado del tipo de la tarjeta
        enum Tipo{Otro, VISA, Mastercard, Maestro, JCB, AmericanExpress};

        //Metodos Observadores
        const Numero& numero() const noexcept;
        const Usuario* titular() const noexcept;
        const Fecha& caducidad() const noexcept;
        Tipo tipo() const noexcept;
        bool activa() const noexcept;
        //Metodo para activar/desactivar la tarjeta
        bool activa(bool b = true) noexcept;

        //Clases para las excepciones
        class Caducada{
            public:
                Caducada(const Fecha& f);
                const Fecha& cuando() const noexcept;
            private:
                Fecha caducada_;
        };
        class Num_duplicado{
            public:
                Num_duplicado(const Numero& n);
                const Numero& que() const noexcept;
            private:
                Numero numero_;
        };
        class Desactivada{};
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    private:
        const Numero numero_;
        Usuario* const titular_;
        const Fecha caducidad_;
        bool activa_;

        //Solo usuario podra acceder a anula_titular
        friend class Usuario;
        void anula_titular();

        //Todas las tarjetas existentes
        static std::set<Numero> tarjetas_;

};
//Inserción en flujo
std::ostream& operator <<(std::ostream& os, const Tarjeta::Tipo& tip);
std::ostream& operator <<(std::ostream& os, const Tarjeta& t) noexcept;

//Metodos Inline

//Operador comparación
inline bool operator <(const Tarjeta& t1, const Tarjeta& t2) noexcept{
    return t1.numero() < t2.numero();
}

//Observadores
inline const Numero& Tarjeta::numero() const noexcept{
    return numero_;
}
inline const Usuario* Tarjeta::titular() const noexcept{
    return titular_;
}
inline const Fecha& Tarjeta::caducidad() const noexcept{
    return caducidad_;
}
inline bool Tarjeta::activa(bool b)noexcept{
    activa_ = b;
    return activa_;
}
inline bool Tarjeta::activa() const noexcept{
    return activa_;
}

//Constructores y observadores de las clases de excepción
inline Tarjeta::Caducada::Caducada(const Fecha& f): caducada_{f}{}
inline const Fecha& Tarjeta::Caducada::cuando() const noexcept{return cad
ucada_;}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
inline Tarjeta::Num_duplicado::Num_duplicado(const Numero& n): numero_{n}
{}
inline const Numero& Tarjeta::Num_duplicado::que() const noexcept{return
numero_;}

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Tarjeta.cpp:

```
#include "tarjeta.hpp"


//Declaraciones anticipadas
std::set<Numero> Tarjeta::tarjetas_;
bool luhn(const Cadena&);

//Metodos Clase Numero-----------------------------------------------------
-------------------------
struct EsDigito{
    bool operator()(char c)const {return isdigit(c);}
    typedef char argument_type;
};

//Constructor
Numero::Numero(const Cadena& cad){
    Cadena n = cad;
    //Suprimimos espacios
    auto it1 = std::remove_if(n.begin(), n.end(), [](char x){return std::
isspace(x);});
    if(it1 != n.end()){
        int cont = 0;
        for(auto j = n.begin(); j != it1; j++, cont++);

        n = n.substr(0,cont);
    }
    //Comprobamos que sean dígitos
    auto it2 = std::find_if(n.begin(), n.end(), std::not1(EsDigito()));
    if(it2 != n.end()){
        Numero::Incorrecto incorr_dig(Razon::DIGITOS);
        throw incorr_dig;
    }
    //Longitud incorrecta --> ERROR
    if (n.length() < 13 || n.length() > 19){
        Numero::Incorrecto incorr_long(Razon::LONGITUD);
        throw incorr_long;
    }else if(!luhn(n)){//Algoritmo de Luhn falla --> ERROR
        Numero::Incorrecto incorr_novalid(Razon::NO_VALIDO);
        throw incorr_novalid;
    }else
        //Copiamos la cadena sin espacios ni caracteres distintos de digi
tos en el atributo de Numero
        numero_ = n;
}

//Métodos de la Clase Tarjeta-----------------------------------------------
-------------------------
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
//Constructor
Tarjeta::Tarjeta(const Numero& n, Usuario& u, const Fecha& f):numero_{n},
 titular_{&u}, caducidad_{f}, activa_{true}{
    Fecha hoy;
    //Fecha de caducidad anterior a la fecha actual --> ERROR
    if(hoy  > f){
        Tarjeta::Caducada caduc(f);
        throw caduc;
    }else if(!tarjetas_.insert(n).second){//Tarjeta ya en el sistema --
> ERROR
        Tarjeta::Num_duplicado numdup(n);
        throw numdup;
    }else //Asignamos la tarjeta al titular
        titular_->es_titular_de(*this);
}

//Destructor
Tarjeta::~Tarjeta(){
    //Si no hemos desligado la tarjeta de su titular aun, lo hacemos
    if(titular_)
        titular_->no_es_titular_de(*this);
    //Borramos la tarjeta
    tarjetas_.erase(numero_);
}

//Metodo Observador del tipo de la tarjeta
Tarjeta::Tipo Tarjeta::tipo()const noexcept{
    //Tomamos los 2 primeros caracteres
    Cadena primeros(2);
    primeros[0] = numero_[0];
    primeros[1] = numero_[1];
    //Segun sean estos dos caracteres, devolvemos un valor del enum Tipo
    if(primeros == "34" || primeros == "37")
        return Tarjeta::AmericanExpress;
    else if(primeros[0] == '3')
        return Tarjeta::JCB;
    else if(primeros[0] == '4')
        return Tarjeta::VISA;
    else if(primeros[0] == '5')
        return Tarjeta::Mastercard;
    else if(primeros[0] == '6')
        return Tarjeta::Maestro;
    else
        return Tarjeta::Otro;

}
//Metodo que solo puede llamar usuario (friend)
void Tarjeta::anula_titular(){
    //Desactivamos la tarjeta
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    activa_=false;
    //La desligamos su titular
    (Usuario*&)titular_ = nullptr;
}

//Insercíon en flujo: Tipo
std::ostream& operator <<(std::ostream& os, const Tarjeta::Tipo& tip){
    //cadena representando el tipo segun sea el tipo de la tarjeta
    switch(tip){
        case Tarjeta::Otro: os << "Tipo indeterminado"; break;
        case Tarjeta::VISA: os << "VISA"; break;
        case Tarjeta::Mastercard:   os << "MasterCard"; break;
        case Tarjeta::Maestro:  os << "Maestro"; break;
        case Tarjeta::JCB:  os << "JCB"; break;
        case Tarjeta::AmericanExpress:  os << "AmericanExpress"; break;
        default: os <<"Error Tipo Tarjeta";
    }
    return os;
}

//Insercion en flujo: Tarjeta
std::ostream& operator <<(std::ostream& os, const Tarjeta& t) noexcept{
    os << " " << std::setfill('-') << std::setw(30) <<  " " << std::endl;
    os << "/                              \\" << std::endl;

    //tipo y numero
    os << "| " << std::left << std::setfill(' ') << std::setw(28) << t.ti
po() << "|" << std::endl;
    os << "| " << std::setfill(' ') << std::setw(28) << t.numero() << "|"
 << std::endl;
    //Cadena auxiliar para mostrar el nombre en mayusculas
    Cadena cad_mostrar = t.titular()->nombre() + " " + t.titular()-
>apellidos();
    for(auto& x:cad_mostrar)
        if(islower(x))
            x = toupper(x);

    os << "| "  << std::setfill(' ') << std::setw(28) << cad_mostrar << "
|" << std::endl;
    //Fecha de vencimiento formateada
    os  << std::right << "| Caduca: " << std::setfill('0') << std::setw(2
) << (t.caducidad().mes()) << "/" << std::setw(2) << (t.caducidad().anno(
)%100) << std::setfill(' ') << std::setw(16) << "|" << std::endl;

    os << "\\                              /" << std::endl;
    os << " ----------------------------" << std::endl;
    return os;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Usuario.hpp:

```
#ifndef _USUARIO_HPP_
#define _USUARIO_HPP_

#include "articulo.hpp"
#include "cadena.hpp"
#include "tarjeta.hpp"

#include <map>              //mapa de tarjetas
#include <unordered_map>    //mapa de articulos
#include <unordered_set>    //set de usuarios
#include <cstdlib>          //rand
#include <unistd.h>         //crypto
#include <cstring>          //strcmp

//Declaraciones anticipadas
class Numero;
class Tarjeta;

//Clase Clave-------------------------------------------------------------
-------------------------
class Clave{
    public:
        //Constructor
        Clave(const char* c);
        //Metodos observadores
        Cadena clave() const noexcept;
        bool verifica(const char* c) const;

        //Enum para excepciones
        enum Razon{CORTA, ERROR_CRYPT};

        //Clase de excepcion
        class Incorrecta{
            public:
                Incorrecta(const Clave::Razon r);
                Clave::Razon razon() const noexcept;
            private:
                Clave::Razon razon_;
        };
    private:
        Cadena clave_;

};

//Metodos inline

//Observador
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
inline Cadena Clave::clave() const noexcept {return clave_;}
//Metodos de la clase de excepcion
inline Clave::Incorrecta::Incorrecta(const Clave::Razon r): razon_{r}{}
inline Clave::Razon Clave::Incorrecta::razon() const noexcept{return razo
n_;}

//Clase Usuario-------------------------------------------------------
-------------------------
class Usuario{
    public:
        //Tipos: Tarjetas y Articulos
        typedef std::map<Numero, Tarjeta*> Tarjetas;
        typedef std::unordered_map<Articulo*, unsigned int> Articulos;

        //Constructores (suprimimos los de copia)
        Usuario(const Cadena& i, const Cadena& n, const Cadena& a, const
Cadena& d, const Clave& c);
        Usuario(const Usuario& u) = delete;
        Usuario& operator =(const Usuario& u) = delete;
        //Destructor
        ~Usuario();

        //Observadores
        const Cadena& id() const noexcept;
        const Cadena& nombre() const noexcept;
        const Cadena& apellidos() const noexcept;
        const Cadena& direccion() const noexcept;
        const Tarjetas& tarjetas() const noexcept;
        const Articulos& compra() const noexcept;
        size_t n_articulos() const noexcept;

        //Metodos para relacionar/borrar relacion con objetos Tarjeta
        void es_titular_de(Tarjeta& t);
        void no_es_titular_de(Tarjeta& t);
        //Metodo para relacionar/borrar relacion con objetos Articulo
        void compra(Articulo& art, size_t cant = 1);

        //Clase de excepcion
        class Id_duplicado{
            public:
                Id_duplicado(const Cadena& i);
                const Cadena& idd() const noexcept;
            private:
                Cadena idd_;
        };
        //Para imprimir la clave, el operador de insercion de flujo debe
ser friend
        friend std::ostream& operator <<(std::ostream& os, const Usuario&
 u) noexcept;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    private:

        const Cadena id_, nombre_, apellidos_, direccion_;
        Clave clave_;
        Tarjetas tarjetas_;
        Articulos articulos_;

        //Set que guarda todos los usuarios
        static std::unordered_set<Cadena> usuarios_;

};
//Mostrar los articulos que tiene en el carro el usuario u
void mostrar_carro(std::ostream& os, const Usuario& u) noexcept;

//Metodos Inline
inline const Cadena& Usuario::id() const noexcept{return id_;}
inline const Cadena& Usuario::nombre() const noexcept{return nombre_;}
inline const Cadena& Usuario::apellidos() const noexcept{return apellidos
_;}
inline const Cadena& Usuario::direccion() const noexcept{return direccion
_;}
inline const Usuario::Tarjetas& Usuario::tarjetas() const noexcept{return
 tarjetas_;}
inline const Usuario::Articulos& Usuario::compra() const noexcept{return
articulos_;}
inline size_t Usuario::n_articulos() const noexcept{return articulos_.siz
e();}

//Metodos Inline de la clase de excepcion
inline Usuario::Id_duplicado::Id_duplicado(const Cadena& i):idd_(i){}
inline const Cadena& Usuario::Id_duplicado::idd() const noexcept{return i
dd_;}

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Usuario.cpp:

```
#include "usuario.hpp"


//Declaracion adelantada
std::unordered_set<Cadena> Usuario::usuarios_;

//Metodos de la Clase Clave-------------------------------------------
-------------------------

//Constructor
Clave::Clave(const char* c){
    char* cad;
    //Cadena corta --> ERROR
    if(strlen(c) < 5) {
        Clave::Incorrecta corta(Clave::CORTA);
        throw corta;
    }
    else{
        //Caracteres disponibles para encriptar
        const char* cjto_caracteres = "./abcdefghijlmnopqrstuvwxyzABCDEFG
HIJKLMNOPQRSTUVWXYZ0123456789";
        //Creamos salt
        char salt[2];
        salt[0] = cjto_caracteres[rand() % strlen(cjto_caracteres)];
        salt[1] = cjto_caracteres[rand() % strlen(cjto_caracteres)];

        //Comprobamos que la encriptacion se haga correctamente, si no es
 asi --> ERROR
        if((cad = (crypt(c,salt))) == NULL){
            Clave::Incorrecta cryp(Clave::ERROR_CRYPT);
            throw cryp;
        }else //Asignamos al atributo la clave encriptada
            clave_ = cad;
    }


}

//Comparacion de una cadena de bajo nivel introducida con una ya cifrada
bool Clave::verifica(const char* c) const{
    return !strcmp(crypt(c, clave_.c_str()), clave_.c_str());
}

//Metodos de la clase Usuario-----------------------------------------
-------------------------

//Constructor
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
Usuario::Usuario(const Cadena& i, const Cadena& n, const Cadena& a, const
 Cadena& d, const Clave& c): id_{i}, nombre_{n}, apellidos_{a}, direccion
_{d}, clave_{c}{
    //Usuario ya en el sistema --> ERROR
    if(!usuarios_.insert(i).second){
        Usuario::Id_duplicado iddup(i);
        throw iddup;
    }
}

//Destructor
Usuario::~Usuario(){
    //Anulamos todas las tarjetas
    for(auto& kv:tarjetas_)
        kv.second->anula_titular();
    //Borramos al usuario del set de usuarios
    usuarios_.erase(id_);
}

//Vincular Usuario--Tarjeta
void Usuario::es_titular_de(Tarjeta& t){
    //Comprobamos que la tarjeta sea del usuario
    if(t.titular() == this)
        //La añadimos a las tarjetas del usuario
        tarjetas_[t.numero()] = &t;
}

//Desvincular Usuario--Tarjeta
void Usuario::no_es_titular_de(Tarjeta& t){
    //Borramos la tarjeta de lasque almacena el usuario (la tarjeta sigue
 <viva>)
    tarjetas_.erase(t.numero());
    t.anula_titular();
    t.activa(true);
}

//Comprar articulo (Vincular/Desvincular Usuario--Articulo)
void Usuario::compra(Articulo& art, size_t cant){
    //Buscamos el articulo en el carrito
    auto en_cesta = articulos_.find(&art);
    //No esta en el carrito --
> Añadimos el articulo si sus unidades son distintas de 0
    if(en_cesta == articulos_.end()){//Nuevo articulo
        if(cant > 0)
            articulos_[&art] = cant;

    }else{//Articulo ya en cesta
        if(cant > 0)//Sumamos cantidades
            en_cesta->second = cant;
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        else //Borramos articulo
            articulos_.erase(en_cesta);
    }
}

//Operador de insercion en flujo
std::ostream& operator <<(std::ostream& os, const Usuario& u) noexcept{
    //datos del usuario
    os << u.id_ << " [" << u.clave_.clave() << "] "  << u.nombre_ << " "
<< u.apellidos_ << std::endl;
    os << u.direccion_ << std::endl;
    //tarjetas
    os << "Tarjetas: " << std::endl;
    for(auto& i:u.tarjetas_)
        os << *i.second << std::endl;
    return os;
}

//Mostrar los articulos que aparecen en el atributo articulos_
void mostrar_carro(std::ostream& os, const Usuario& u) noexcept{
    //Para usar la coma en vez del punto en el precio
    std::locale::global(std::locale(""));
    //Datos del usuario y numero de elementos en el carrito
    os << "Carrito de la compra de " << u.id() << " [Artículos: " << u.n_
articulos() << "]" << std::endl;
    os << "Cant.\tArt." <<std::endl;
    os << "================================================================
=====================" << std::endl;
    //elementos y cantidades
    for(auto& i:u.compra()){
        //Cantidad
        os << i.second << "\t";
        //[REF] "titulo". anno. precio con coma en euros
        os << "[" << i.first->referencia() << "] " << "\"" << i.first-
>titulo() << "\", " << i.first-
>f_publi().anno() << ". " << std::fixed << std::setprecision(2) << i.firs
t->precio() << " €" << std::endl;
    }
    os << std::endl;

}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Pedido.hpp:

```
#ifndef _PEDIDO_HPP_
#define _PEDIDO_HPP_

#include <iostream>
#include "tarjeta.hpp"
#include "pedido-articulo.hpp"
#include "usuario-pedido.hpp"
#include "articulo.hpp"

class Usuario;
class Fecha;
class Articulo;
class Usuario_Pedido;
class Pedido_Articulo;

class Pedido{
    public:
        //Constructor
        Pedido(Usuario_Pedido& up, Pedido_Articulo& pa, Usuario& u, const
 Tarjeta& t, const Fecha& f = Fecha());
        //Observadores
        const int numero() const noexcept;
        const Tarjeta* tarjeta() const noexcept;
        const Fecha& fecha() const noexcept;
        const double total() const noexcept;
        static const int n_total_pedidos() noexcept;

        //Clases de excepcion
        class Vacio{
            public:
                Vacio(const Usuario& us);
                const Usuario& usuario() const;
            private:
                const Usuario* u;
        };
        class Impostor{
            public:
                Impostor(const Usuario& us);
                const Usuario& usuario() const;
            private:
                const Usuario* u;
        };
        class SinStock{
            public:
                SinStock(const Articulo& art);
                const Articulo& articulo() const;
            private:
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
                const Articulo* a;
        };
    private:
        int num;
        const Tarjeta* tarjeta_;
        Fecha fecha_;
        double total_;
        static int N_pedidos;


};
//Insercion en flujo
std::ostream& operator <<(std::ostream& os, const Pedido& p);

//Funciones Inline (clases de excepcion y observadores)
inline Pedido::Vacio::Vacio(const Usuario& us): u{&us}{};
inline const Usuario& Pedido::Vacio::usuario() const{return *u;}
inline Pedido::Impostor::Impostor(const Usuario& us): u{&us}{};
inline const Usuario& Pedido::Impostor::usuario() const{return *u;}
inline Pedido::SinStock::SinStock(const Articulo& art): a{&art}{};
inline const Articulo& Pedido::SinStock::articulo() const{return *a;}

inline const int Pedido::numero() const noexcept{return num;}
inline const Tarjeta* Pedido::tarjeta() const noexcept{return tarjeta_;}
inline const Fecha& Pedido::fecha() const noexcept{return fecha_;}
inline const double Pedido::total() const noexcept{return total_;}
inline const int Pedido::n_total_pedidos() noexcept{return N_pedidos;}

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Pedido.cpp:

```
#include "pedido.hpp"
#include <unordered_map>


//Valor inicial de n_pedidos: cero
int Pedido::N_pedidos = 0;
//Constructor Pedido
Pedido::Pedido(Usuario_Pedido& up, Pedido_Articulo& pa, Usuario& u, const
 Tarjeta& t, const Fecha& f): num{N_pedidos+1}, tarjeta_{&t}, fecha_{f},
total_{0.}{
    //Tarjeta desactivada
    if(!t.activa()){
        throw Tarjeta::Desactivada{};
    }
    //Tarjeta caducada
    if(f > t.caducidad()){
        Tarjeta::Caducada tarjcad(t.caducidad());
        throw tarjcad;
    }
    //Tarjeta no pertenece al usuario
    if(&u != t.titular()){
        Pedido::Impostor sus(u);
        throw sus;
    }



    auto i = u.compra().begin();
    while(i != u.compra().end()){
        //En un articulo almacenabe
        if(ArticuloAlmacenable* AA = dynamic_cast<ArticuloAlmacenable*>(i
->first)){
            //Comprobamos que el stock de cada articulo almacenable del c
arrito no sea insuficiente
            if(AA->stock() < i->second){
                Pedido::SinStock nostock{*i->first};
                const_cast<Usuario::Articulos&>(u.compra()).clear();
                throw nostock;
            }
            i++;
        }
        else if(LibroDigital* LD = dynamic_cast<LibroDigital*>(i-
>first)){//Es un libro digital
            //Comprobamos que la fecha de expiracion sea posterior a la d
el pedido
            if(LD->f_expir() < f){
                //Si borramos del carrito el articulo perdemos el iterado
r, asi que antes
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
                //de eliminarlo guardamos el siguiente para poder continu
ar el bucle
                auto aux = i;
                aux++;
                u.compra(*i->first, 0);
                i = aux;
            }else{
                i++;
            }
        }
        else{
            throw std::logic_error("Tipo desconocido");
        }
    }

    //Carrito vacio
    if(u.n_articulos() == 0){
        Pedido::Vacio vac{u};
        throw vac;
    }
    //Creamos la asociacion entre usuario y pedido
    up.asocia(u,*this);
    //Vamos decrementando el stock de cada articulo, sumamos la cantidad
de precio al total,
    //creamos la asociacion entre el articulo y el pedido y borramos el a
rticulo del carrito
    while(!u.compra().empty()){
        auto i = u.compra().begin();
        if(ArticuloAlmacenable* AA = dynamic_cast<ArticuloAlmacenable*>(i
->first)){
            AA->stock() -= i->second;
        }
        total_ += i->second * i->first->precio();
        pa.pedir(*i->first, *this, i->first->precio(), i->second);
        u.compra(*i->first, 0);
    }
    //Incrementamos el numero de pedidos
    N_pedidos++;
}
//Insercion en flujo PEDIDO
std::ostream& operator <<(std::ostream& os, const Pedido& p){
    os << "Núm. pedido: " << p.numero() << std::endl;
    os << "Fecha: " << p.fecha() << std::endl;
    os << "Pagado con: " << p.tarjeta()-
>tipo() << " n.º:" << p.tarjeta()->numero() << std::endl;
    os << "Importe: " << p.total() << " €" << std::endl;
    return os;
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Usuario-Pedido.hpp:

```
#ifndef _USUARIO_PEDIDO_HPP_
#define _USUARIO_PEDIDO_HPP_

#include <set>
#include <map>

class Usuario;
class Pedido;

class Usuario_Pedido{
    public:
        //Tipo Pedidos
        typedef std::set<Pedido*> Pedidos;
        //Asociacion
        void asocia(Usuario& u, Pedido& p);
        void asocia(Pedido& p, Usuario& u);
        //Observadores
        Pedidos pedidos(const Usuario& u)const noexcept;
        const Usuario* cliente(const Pedido& u) const noexcept;

    private:
        //Diccionarios
        typedef std::map<Usuario*, Pedidos> UsuPed;
        typedef std::map<Pedido*, Usuario*> PedUsu;

        UsuPed up;
        PedUsu pu;
};

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Usuario-Pedido.cpp:

```
#include "usuario-pedido.hpp"

//Asociacion-------------------------------------------------------------
-------------------------
void Usuario_Pedido::asocia(Usuario& u, Pedido& p){
    //Añadimos al dicionario los valores
    up[&u].insert((Pedido*)&p);
    pu[&p] = (Usuario*)&u;
}

//En la función sobrecargada se llama a la otra funcion asocia
void Usuario_Pedido::asocia(Pedido& p, Usuario& u){
    asocia(u,p);
}

//Observadores-----------------------------------------------------------
-------------------------
Usuario_Pedido::Pedidos Usuario_Pedido::pedidos(const Usuario& u)const no
except{
    //Buscamos el usuario en el diccionario
    auto i = up.find((Usuario*)&u);
    if(i != up.end()) //Esta en el diccionario
        //Devolvemos el set asociado al usuario
        return i->second;
    else //No estaba en el diccionario
        //Devolvemos un set vacio
        return Usuario_Pedido::Pedidos{};
}

const Usuario* Usuario_Pedido::cliente(const Pedido& p) const noexcept{
    //Buscamos en el diccionario
    auto i = pu.find((Pedido*)&p);
    if(i != pu.end())//Esta en el diccionario
        //Devolvemos el usuario asociado
        return i->second;
    else{ //No esta en el diccionario
        //Devolvemos un puntero vacio
        return nullptr;
    }
}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Pedido-Articulo.hpp:

```
#ifndef _PEDIDO_ARTICULO_HPP_
#define _PEDIDO_ARTICULO_HPP_
#include <iostream>
#include <map>
#include <iomanip>


#include "pedido.hpp"
#include "articulo.hpp"

class Pedido;

//LINEAPEDIDO-------------------------------------------------------------
-------------------------
class LineaPedido{
    public:
        //Constructor
        explicit LineaPedido(double p, unsigned cantidad = 1);
        //Observadores
        const double precio_venta() const noexcept;
        const unsigned cantidad() const noexcept;

    private:
        double precio_venta_;
        unsigned cantidad_;
};
//Insercion en flujo
std::ostream& operator <<(std::ostream& os, const LineaPedido& lp);

//Funciones inline de lineapedido
inline LineaPedido::LineaPedido(double p, unsigned cant): precio_venta_{p
}, cantidad_{cant}{}
inline const double LineaPedido::precio_venta() const noexcept {return pr
ecio_venta_;}
inline const unsigned LineaPedido::cantidad() const noexcept {return cant
idad_;}

//ORDENACION--------------------------------------------------------------
-------------------------

class OrdenaArticulos: public std::binary_function<Articulo*, Articulo*,
bool>{
    public:
        bool operator()(const Articulo* a1, const Articulo* a2)const noex
cept;
};

class OrdenaPedidos: public std::binary_function<Pedido*, Pedido*, bool>{
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
    public:
        bool operator()(const Pedido* p1, const Pedido* p2)const noexcept
;
};


//PEDIDOARTICULO-------------------------------------------------------
-------------------------
class Pedido_Articulo{
    public:
        //Tipos definidos
        typedef std::map<Articulo*, LineaPedido, OrdenaArticulos> ItemsPe
dido;
        typedef std::map<Pedido*, LineaPedido, OrdenaPedidos> Pedidos;
        //Metodos de asociacion
        void pedir(Pedido& ped, Articulo& art, double pr, unsigned cant =
 1);
        void pedir(Articulo& art, Pedido& ped, double pr, unsigned cant =
 1);
        //Observadores
        ItemsPedido detalle(const Pedido& ped) const;
        Pedidos ventas(const Articulo& art) const;
        //Metodos mostrar
        void mostrarDetallePedidos(std::ostream& os) const noexcept;
        void mostrarVentasArticulos(std::ostream& os) const noexcept;
    private:
        //Diccionarios
        typedef std::map<Pedido*, ItemsPedido, OrdenaPedidos> PedArt;
        typedef std::map<Articulo*, Pedidos, OrdenaArticulos> ArtPed;
        PedArt pa;
        ArtPed ap;
};
//Insercion en flujo
std::ostream& operator <<(std::ostream& os, const Pedido_Articulo::ItemsP
edido& ip);
std::ostream& operator <<(std::ostream& os, const Pedido_Articulo::Pedido
s& p);

#endif
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

·Pedido-Articulo.cpp:

```
#include "pedido-articulo.hpp"

//LINEAPEDIDO-----------------------------------------------------------
-------------------------
//Insercion en flujo LINEAPEDIDO
std::ostream& operator <<(std::ostream& os, const LineaPedido& lp){
    std::locale::global(std::locale(""));
    os << std::fixed << std::setprecision(2) << lp.precio_venta() << " €\
t" << lp.cantidad();
    return os;
}

//PEDIDO----------------------------------------------------------------
-------------------------
//Metodos de asociacion
void Pedido_Articulo::pedir(Pedido& ped, Articulo& art, double pr, unsign
ed cant){
    pa[&ped].insert(std::make_pair(&art, LineaPedido(pr,cant)));
    ap[&art].insert(std::make_pair(&ped,LineaPedido(pr,cant)));

}
//Sobrecarga: Llama al otro metodo pedir
void Pedido_Articulo::pedir(Articulo& art, Pedido& ped, double pr, unsign
ed cant){
    pedir(ped,art,pr,cant);
}
//Observadores
Pedido_Articulo::ItemsPedido Pedido_Articulo::detalle(const Pedido& ped)
const{
    //Busqueda en el diccionario
    auto i = pa.find((Pedido*)&ped);
    if(i != pa.end())//Existe en el diccionario
        //Devuelve ItemsPedido asociado
        return i->second;
    else
        //Devuelve temsPedido vacio
        return ItemsPedido{};
}

Pedido_Articulo::Pedidos Pedido_Articulo::ventas(const Articulo& art) con
st{
    //Busqueda en el diccionario
    auto i = ap.find((Articulo*)&art);
    if(i != ap.end())//Existe en el diccionario
        //Devuelve los pedidos asociados
        return i->second;
    else
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
        //Devuelve Pedidos vacio
        return Pedidos{};
}
//Insercion en flujo ItemsPedido
std::ostream& operator <<(std::ostream& os, const Pedido_Articulo::ItemsP
edido& ip){
    //Para calcular el total
    double total = 0.;
    os << " PVP\tCantidad\tArtículo" << std::endl;
    os << std::setfill('=') << std::setw(75) << " " << std::endl;
    //Imprimimos todo el contenido
    for(auto& i:ip){
        total += (i.second.precio_venta() * i.second.cantidad());
        os << i.second << "\t\t";
        os << "[" << i.first->referencia() << "] " << i.first-
>titulo() << std::endl;;

    }
    os << std::setfill('=') << std::setw(75) << " " << std::endl;
    os << "Total\t" << total << " €" << std::endl << std::endl << std::en
dl;
    return os;
}
//Insercion en flujo Pedidos
std::ostream& operator <<(std::ostream& os, const Pedido_Articulo::Pedido
s& p){
    //Para calcular el total de precio y de cantidad de articulos
    double total = 0.;
    unsigned cantidad = 0;
    os << "[Pedidos: " << p.size() << "]" << std::endl;
    os << std::setfill('=') << std::setw(75) << " " << std::endl;
    os << " PVP\tCantidad\tFecha" << std::endl;
    os << std::setfill('=') << std::setw(75) << " " << std::endl;
    //Imprimimos todo el contenido
    for(auto& i:p){
        total += i.second.precio_venta() * i.second.cantidad();
        cantidad += i.second.cantidad();
        os << i.second << "\t\t" << i.first->fecha() << std::endl;
    }
    os << std::setfill('=') << std::setw(75) << " " << std::endl;
    os << total << " €\t" << cantidad << std::endl << std::endl << std::e
ndl;
    return os;
}


//Funciones mostrar
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

```
void Pedido_Articulo::mostrarDetallePedidos(std::ostream& os) const noexc
ept{
    //Para calcular el total ventas
    double precio = 0.;
    for(auto& i:pa){
        os << "Pedido núm. " << i.first->numero() << std::endl;
        os << "Cliente: " << i.first->tarjeta()->titular()-
>id() << "\tFecha: " << i.first->fecha() << std::endl;
        //Imprimimos ItemsPedido
        os << detalle(*i.first);
        precio += i.first->total();
    }
    os << std::endl << "TOTAL VENTAS\t" << precio << " €" << std::endl;
}

void Pedido_Articulo::mostrarVentasArticulos(std::ostream& os) const noex
cept{
    for(auto& i:ap){
        os << "Ventas de " << "[" << i.first-
>referencia() << "] " << i.first->titulo() << std::endl;
        //Imprimimos Pedidos
        os << ventas(*i.first);
    }
}
//-----------------------------------------------------------------------
-------------------------
bool OrdenaArticulos::operator()(const Articulo* a1, const Articulo* a2)c
onst noexcept{return a1->referencia() < a2->referencia();}
bool OrdenaPedidos::operator()(const Pedido* p1, const Pedido* p2)const n
oexcept{return p1->numero() < p2->numero();}
```


WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.

WUOLAH

Reservados todos los derechos. No se permite la explotación económica ni la transformación de esta obra. Queda permitida la impresión en su totalidad.